

Red Hat OpenShift Data Foundation 4.22

Managing and allocating storage resources

Instructions on how to allocate storage to core services and hosted applications in OpenShift Data Foundation, including snapshot and clone.

Last Updated: 2026-09-21

Red Hat OpenShift Data Foundation 4.22 Managing and allocating storage resources

Instructions on how to allocate storage to core services and hosted applications in OpenShift Data Foundation, including snapshot and clone.

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack ® Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

This document explains how to allocate storage to core services and hosted applications in Red Hat OpenShift Data Foundation.

Table of Contents

PREFACE 5

PROVIDING FEEDBACK ON RED HAT DOCUMENTATION 6

CHAPTER 1. OVERVIEW 7

CHAPTER 2. STORAGE CLASSES 8
2.1. CREATING STORAGE CLASSES AND POOLS 8
2.2. STORAGE CLASS WITH SINGLE REPLICA 9
2.2.1. Recovering after OSD lost from single replica 14

CHAPTER 3. PERSISTENT VOLUME ENCRYPTION 16
3.1. ACCESS CONFIGURATION FOR KEY MANAGEMENT SYSTEM (KMS) 16
3.1.1. Configuring access to KMS using vaulttokens 16
3.1.2. Configuring access to KMS using Thales CipherTrust Manager 17
3.1.3. Configuring access to KMS using vaulttenantsa 18
3.2. CREATING A STORAGE CLASS FOR PERSISTENT VOLUME ENCRYPTION 21
3.2.1. Overriding Vault connection details using tenant ConfigMap 25
3.3. ENABLING AND DISABLING KEY ROTATION WHEN USING KMS 25
3.3.1. Enforcing precedence for key rotation 26
3.3.2. Enabling key rotation 26
3.3.2.1. Annotating a namespace for key rotation 26
3.3.2.2. Annotating a storage class for key rotation 27
3.3.2.3. Annotating a PersistentVolumeClaim for key rotation 28
3.3.3. Disabling key rotation 28

CHAPTER 4. ENABLING AND DISABLING ENCRYPTION IN-TRANSIT POST DEPLOYMENT 30
4.1. ENABLING ENCRYPTION IN-TRANSIT AFTER DEPLOYMENT IN INTERNAL MODE 30
4.2. DISABLING ENCRYPTION IN-TRANSIT AFTER DEPLOYMENT IN INTERNAL MODE 31
4.3. ENABLING ENCRYPTION IN-TRANSIT AFTER DEPLOYMENT IN EXTERNAL MODE 32
4.4. DISABLING ENCRYPTION IN-TRANSIT AFTER DEPLOYMENT IN EXTERNAL MODE 34

CHAPTER 5. BLOCK POOLS 38
5.1. MANAGING BLOCK POOLS IN INTERNAL MODE 38
5.1.1. Creating a block pool 38
5.1.2. Updating an existing pool 39
5.1.3. Deleting a pool 39

CHAPTER 6. CONFIGURE STORAGE FOR OPENSHIFT CONTAINER PLATFORM SERVICES 41
6.1. CONFIGURING IMAGE REGISTRY TO USE OPENSHIFT DATA FOUNDATION 41
6.2. USING MULTICLOUD OBJECT GATEWAY AS OPENSHIFT IMAGE REGISTRY BACKEND STORAGE 43
6.3. CONFIGURING MONITORING TO USE OPENSHIFT DATA FOUNDATION 46
6.4. OVERPROVISION LEVEL POLICY CONTROL 48
6.5. CLUSTER LOGGING FOR OPENSHIFT DATA FOUNDATION 50

CHAPTER 7. BACKING OPENSHIFT CONTAINER PLATFORM APPLICATIONS WITH OPENSHIFT DATA FOUNDATION 52

CHAPTER 8. ADDING FILE AND OBJECT STORAGE TO AN EXISTING EXTERNAL OPENSHIFT DATA FOUNDATION CLUSTER 54

CHAPTER 9. HOW TO USE DEDICATED WORKER NODES FOR RED HAT OPENSHIFT DATA FOUNDATION 58
9.1. ANATOMY OF AN INFRASTRUCTURE NODE 58
9.2. MACHINE SETS FOR CREATING INFRASTRUCTURE NODES 58

9.3. MANUAL CREATION OF INFRASTRUCTURE NODES 59
9.4. TAINTING A NODE FROM THE USER INTERFACE 60

CHAPTER 10. MANAGING PERSISTENT VOLUME CLAIMS 62
10.1. CONFIGURING APPLICATION PODS TO USE OPENSHIFT DATA FOUNDATION 62
10.2. VIEWING PERSISTENT VOLUME CLAIM REQUEST STATUS 63
10.3. REVIEWING PERSISTENT VOLUME CLAIM REQUEST EVENTS 64
10.4. EXPANDING PERSISTENT VOLUME CLAIMS 64
10.5. DYNAMIC PROVISIONING 66
10.5.1. About dynamic provisioning 66
10.5.2. Dynamic provisioning in OpenShift Data Foundation 66
10.5.3. Available dynamic provisioning plug-ins 67

CHAPTER 11. RECLAIMING SPACE ON TARGET VOLUMES 69
11.1. ENABLING RECLAIM SPACE OPERATION BY ANNOTATING PERSISTENTVOLUMECLAIMS 69
11.2. DISABLING RECLAIM SPACE FOR A SPECIFIC PERSISTENTVOLUMECLAIM 71
11.3. ENABLING RECLAIM SPACE OPERATION USING RECLAIMSPACEJOB 71
11.4. ENABLING RECLAIM SPACE OPERATION USING RECLAIMSPACECRONJOB 72
11.5. CUSTOMISING TIMEOUTS REQUIRED FOR RECLAIM SPACE OPERATION 74
11.6. ENFORCING PRECEDENCE FOR RECLAIM SPACE OPERATIONS 75

CHAPTER 12. FINDING AND CLEANING STALE SUBVOLUMES 76
12.1. CLEANING ORPHANED CEPHFS SNAPSHOTS 76

CHAPTER 13. VOLUME SNAPSHOTS 78
13.1. CREATING VOLUME SNAPSHOTS 78
13.2. RESTORING VOLUME SNAPSHOTS 79
13.3. DELETING VOLUME SNAPSHOTS 80

CHAPTER 14. VOLUME CLONING 82
14.1. CREATING A CLONE 82

CHAPTER 15. MANAGING CONTAINER STORAGE INTERFACE (CSI) COMPONENT PLACEMENTS 83

CHAPTER 16. USING 2-WAY REPLICATION WITH CEPHFS 84
16.1. EDITING THE EXISTING DEFAULT CEPHFS DATA POOL TO REPLICA-2 84
16.2. ADDING AN ADDITIONAL CEPHFS DATA POOL WITH REPLICA-2 85

CHAPTER 17. CREATING EXPORTS USING NFS 86
17.1. ENABLING THE NFS FEATURE 86
17.2. CREATING NFS EXPORTS 87
17.3. CONSUMING NFS EXPORTS IN-CLUSTER 88
17.4. CONSUMING NFS EXPORTS IN OPENSHIFT CLUSTER 90

CHAPTER 18. ANNOTATING ENCRYPTED RBD STORAGE CLASSES 92

CHAPTER 19. ENABLING FASTER CLIENT IO OR RECOVERY IO DURING OSD BACKFILL 93

CHAPTER 20. SETTING CEPH OSD FULL THRESHOLDS 94
20.1. SETTING CEPH OSD FULL THRESHOLDS USING THE ODF CLI TOOL 94
20.2. SETTING CEPH OSD FULL THRESHOLDS BY UPDATING THE STORAGECLUSTER CR 95

CHAPTER 21. CONFIGURING CEPH TARGET SIZE RATIOS 97

CHAPTER 22. CONFIGURING CEPH MONITOR FAILOVER TIMEOUT 100

CHAPTER 23. BACKUP AND RECOVERY OF MULTICLOUD OBJECT GATEWAY METADATA DATABASE

(NOOBAA DB) 102
23.1. ENABLING BACKUP OF MULTICLOUD OBJECT GATEWAY METADATA DATABASE USING STORAGECLUSTER CR 102
23.2. ENABLING BACKUP OF MULTICLOUD OBJECT GATEWAY METADATA DATABASE USING MCG CLI 102
23.3. RECOVERING MULTICLOUD OBJECT GATEWAY METADATA DATABASE FROM BACKUP 103

PREFACE

Learn how to allocate OpenShift Data Foundation storage to core OpenShift Container Platform services and hosted applications.

PROVIDING FEEDBACK ON RED HAT DOCUMENTATION

We appreciate your input on our documentation. Do let us know how we can make it better.

To give feedback, create a Jira ticket:

Procedure

1. Log in to the Jira.
2. Click **Create** in the top navigation bar
3. Enter a descriptive title in the Summary field.
4. Enter your suggestion for improvement in the Description field. Include links to the relevant parts of the documentation.
5. Select **Documentation** in the Components field.
6. Click Create at the bottom of the dialogue.

CHAPTER 1. OVERVIEW

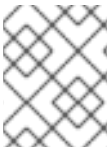
Read this document to understand how to create, configure, and allocate storage to core services or hosted applications in Red Hat OpenShift Data Foundation.

- Storage classes shows you how to create custom storage classes.
- Block pools provides you with information on how to create, update and delete block pools.
- Configure storage for OpenShift Container Platform services shows you how to use OpenShift Data Foundation for core OpenShift Container Platform services.
- Backing OpenShift Container Platform applications with OpenShift Data Foundation provides information about how to configure OpenShift Container Platform applications to use OpenShift Data Foundation.
- Adding file and object storage to an existing external OpenShift Data Foundation cluster
- How to use dedicated worker nodes for Red Hat OpenShift Data Foundation provides information about how to use dedicated worker nodes for Red Hat OpenShift Data Foundation.
- Managing Persistent Volume Claims provides information about managing Persistent Volume Claim requests, and automating the fulfillment of those requests.
- Reclaiming space on target volumes shows you how to reclaim the actual available storage space.
- Volume snapshots shows you how to create, restore, and delete volume snapshots.
- Volume cloning shows you how to create volume clones.
- Managing container storage interface (CSI) component placements provides information about setting tolerations to bring up container storage interface component on the nodes.

CHAPTER 2. STORAGE CLASSES

Storage classes define how persistent volumes are provisioned and behave in your OpenShift Data Foundation deployment. Create custom storage classes to match workload requirements such as specific replication levels, compression, or encryption.

The OpenShift Data Foundation operator installs a default storage class depending on the platform in use. This default storage class is owned and controlled by the operator and it cannot be deleted or modified. However, you can create custom storage classes to use other storage resources or to offer a different behavior to applications.



NOTE

Custom storage classes are not supported for *external mode* OpenShift Data Foundation clusters.

2.1. CREATING STORAGE CLASSES AND POOLS

You can create a storage class using an existing pool or you can create a new pool for the storage class while creating it.

Prerequisites

- Ensure that you are logged into the OpenShift Container Platform web console and OpenShift Data Foundation cluster is in **Ready** state.

Procedure

1. Click **Storage → StorageClasses**.
2. Click **Create Storage Class**
3. Enter the storage class **Name** and **Description**.
4. **Reclaim Policy** is set to **Delete** as the default option. Use this setting.
   If you change the reclaim policy to **Retain** in the storage class, the persistent volume (PV) remains in **Released** state even after deleting the persistent volume claim (PVC).
5. **Volume binding mode** is set to **WaitForConsumer** as the default option.
   If you choose the **Immediate** option, then the PV gets created immediately when creating the PVC.
6. Select **RBD** or **CephFS Provisioner** as the plugin for provisioning the persistent volumes.
7. Choose a **Storage system** for your workloads.
8. Select an existing **Storage Pool** from the list or create a new pool.



NOTE

The 2-way replication data protection policy is only supported for the non-default RBD pool. 2-way replication can be used by creating an additional pool. To know about Data Availability and Integrity considerations for replica 2 pools, see Knowledgebase Customer Solution Article .

Create new pool

a. Click **Create New Pool**

b. Enter **Pool name**.

c. Select **Data protection policy** as either **Replication** or **Erasure coding**.

i. Select **Erasure coding** to enable the use of erasure coding instead of replication to protect the cluster data. Supported only for Block (RBD) and File (CephFS).

ii. Select the **Scheme** (k+m) for your workload.



NOTE

The minimum number of storage nodes required is equal to the number of data chunks (k) plus the number of parity chunks (m). The recommended minimum is k + m + 1 nodes.

d. Select **Data compression** if you need to compress the data.
Enabling compression can impact application performance and might prove ineffective when data to be written is already compressed or encrypted. Data written before enabling compression will not be compressed.

e. Click **Create** to create the new storage pool.

f. Click **Finish** after the pool is created.

9. Optional: Select **Enable Encryption** checkbox.

10. Click **Create** to create the storage class.

2.2. STORAGE CLASS WITH SINGLE REPLICA

You can create a storage class with a single replica to be used by your applications. This avoids redundant data copies and allows resiliency management on the application level.



WARNING

Enabling this feature creates a single replica pool without data replication, increasing the risk of data loss, data corruption, and potential system instability if your application does not have its own replication. If any OSDs are lost, this feature requires very disruptive steps to recover. All applications can lose their data, and must be recreated in case of a failed OSD.

Prerequisites

- Make sure to use one new storage device or OSD per failure domain.

Procedure

1. Enable the single replica feature using the following command:

```
$ oc patch storagecluster ocs-storagecluster -n openshift-storage --type json --patch '[{ "op": "replace", "path": "/spec/managedResources/cephNonResilientPools/enable", "value": true }]'
```

2. Verify **storagecluster** is in **Ready** state:

```
$ oc get storagecluster
```

Example output:

```
NAME                 AGE   PHASE   EXTERNAL   CREATED AT             VERSION
ocs-storagecluster   10m   Ready              2024-02-05T13:56:15Z   4.22.0
```

3. New **cephblockpools** are created for each failure domain. Verify **cephblockpools** are in **Ready** state:

```
$ oc get cephblockpools
```

Example output:

```
NAME                                    PHASE
ocs-storagecluster-cephblockpool        Ready
ocs-storagecluster-cephblockpool-us-east-1a   Ready
ocs-storagecluster-cephblockpool-us-east-1b   Ready
ocs-storagecluster-cephblockpool-us-east-1c   Ready
```

4. Verify new storage classes have been created:

```
$ oc get storageclass
```

Example output:

```
NAME                                    PROVISIONER                     RECLAIMPOLICY
VOLUMEBINDINGMODE      ALLOWVOLUMEEXPANSION   AGE
gp2 (default)                           kubernetes.io/aws-ebs           Delete
WaitForFirstConsumer   true                   104m
gp2-csi                                 ebs.csi.aws.com                 Delete
WaitForFirstConsumer   true                   104m
gp3-csi                                 ebs.csi.aws.com                 Delete
WaitForFirstConsumer   true                   104m
ocs-storagecluster-ceph-non-resilient-rbd   openshift-storage.rbd.csi.ceph.com      Delete
WaitForFirstConsumer   true                   46m
ocs-storagecluster-ceph-rbd             openshift-storage.rbd.csi.ceph.com      Delete
Immediate              true                   52m
ocs-storagecluster-cephfs               openshift-storage.cephfs.csi.ceph.com   Delete
Immediate              true                   52m
openshift-storage.noobaa.io             openshift-storage.noobaa.io/obc         Delete
Immediate              false                  50m
```

5. Three new OSDs are deployed on the new devices after the storage class with single replica is enabled; 3 **osd-prepare** pods and 3 additional pods. Verify new OSD pods are in **Running** state:

```
$ oc get pods | grep osd
```

Example output:

```
rook-ceph-osd-0-6dc76777bc-snhnm                                  2/2     Running     0          9m50s
rook-ceph-osd-1-768bdfdc4-h5n7k                                   2/2     Running     0          9m48s
rook-ceph-osd-2-69878645c4-bkdlq                                  2/2     Running     0          9m37s
rook-ceph-osd-3-64c44d7d76-zfxq9                                  2/2     Running     0          5m23s
rook-ceph-osd-4-654445b78f-nsgjb                                  2/2     Running     0          5m23s
rook-ceph-osd-5-5775949f57-vz6jp                                  2/2     Running     0          5m22s
rook-ceph-osd-prepare-ocs-deviceset-gp2-0-data-0x6t87-59swf       0/1     Completed   0
10m
rook-ceph-osd-prepare-ocs-deviceset-gp2-1-data-0klwr7-bk45t       0/1     Completed   0
10m
rook-ceph-osd-prepare-ocs-deviceset-gp2-2-data-0mk2cz-jx7zv       0/1     Completed   0
10m
```

6. Delete Workloads using the non-resilient(replica-1) storageClass.



NOTE

Disabling replica-1 is not a tested and supported scenario. Perform the following steps only if it is necessary to disable and clean up replica-1.

```
$ oc delete deployment replica-1-workload
deployment.apps "replica-1-workload" deleted
```

1. Delete all the PVCs using the non-resilient storageClass.

```
$ oc get pvc --all-namespaces -o custom-
columns="NAMESPACE:.metadata.namespace,NAME:.metadata.name,STORAGECLASS:.sp
ec.storageClassName" | grep "ocs-storagecluster-ceph-non-resilient-rbd"
```

```
$ oc delete pvc replica-1-pvc
persistentvolumeclaim "replica-1-pvc" deleted
```

2. Mark the non-resilient pools **Enable** flag to false on the storageCluster CR.

```
$ oc patch storagecluster ocs-storagecluster -n openshift-storage --type json --patch '[{ "op":
"replace", "path": "/spec/managedResources/cephNonResilientPools/enable", "value": false
}]'
```

3. Remove the non-resilient-rbd storageClass from storageconsumer CR spec.storageClasses.

```
$ oc get storageconsumer internal -n openshift-storage -o json | \
jq 'del(.spec.storageClasses[] | select(.name == "ocs-storagecluster-ceph-non-resilient-rbd"))'
| \
oc replace -f -
```

4. Delete the non-resilient-rbd storageClass.

```
$ oc delete sc ocs-storagecluster-ceph-non-resilient-rbd
```

5. Delete the replica-1 cephBlockPools.

```
$ oc get cephblockpool
NAME                                         PHASE   TYPE        FAILUREDOMAIN   AGE
builtin-mgr                                  Ready   Replicated  zone            136m
ocs-storagecluster-cephblockpool             Ready   Replicated  zone            136m
ocs-storagecluster-cephblockpool-us-east-1   Ready   Replicated  zone            50m
ocs-storagecluster-cephblockpool-us-east-2   Ready   Replicated  zone            50m
ocs-storagecluster-cephblockpool-us-east-3   Ready   Replicated  zone            50m

$ oc delete cephblockpool ocs-storagecluster-cephblockpool-us-east-1 ocs-storagecluster-
cephblockpool-us-east-2 ocs-storagecluster-cephblockpool-us-east-3
cephblockpool.ceph.rook.io "ocs-storagecluster-cephblockpool-us-east-1" deleted
cephblockpool.ceph.rook.io "ocs-storagecluster-cephblockpool-us-east-2" deleted
cephblockpool.ceph.rook.io "ocs-storagecluster-cephblockpool-us-east-3" deleted
```

6. Remove the replica-1 OSDs.

   a. Identify the replica-1 OSDs with the class name of the failure domain,

```
sh-5.1$ ceph osd df tree
ID   CLASS     WEIGHT   REWEIGHT  SIZE     RAW USE  DATA     OMAP    META
AVAIL    %USE  VAR   PGS  STATUS  TYPE NAME
 -1           1.50000         -  1.5 TiB  4.8 GiB  4.4 GiB  32 KiB  395 MiB  1.5 TiB  0.31  1.00
-          root default
 -5           1.50000         -  1.5 TiB  4.8 GiB  4.4 GiB  32 KiB  395 MiB  1.5 TiB  0.31  1.00
-              region us-east
-14           0.50000         -  512 GiB  1.7 GiB  1.5 GiB  12 KiB  176 MiB  510 GiB  0.32
1.03    -                  zone us-east-1
-13           0.25000         -  256 GiB  1.6 GiB  1.5 GiB  11 KiB   75 MiB  254 GiB  0.61
1.93    -                      host ocs-deviceset-2-data-0kp6fl
  2       ssd  0.25000   1.00000  256 GiB  1.6 GiB  1.5 GiB  11 KiB   75 MiB  254 GiB  0.61
1.93  353      up                  osd.2
-46           0.25000         -  256 GiB  104 MiB  2.7 MiB   1 KiB  101 MiB  256 GiB  0.04
0.13    -                      host us-east-1-data-0c2xxv
  3  us-east-1  0.25000   1.00000  256 GiB  104 MiB  2.7 MiB   1 KiB  101 MiB  256 GiB
0.04  0.13    0      up                  osd.3
 -4           0.50000         -  512 GiB  1.6 GiB  1.5 GiB   8 KiB  101 MiB  510 GiB  0.31  0.98
-                  zone us-east-2
 -3           0.25000         -  256 GiB  1.6 GiB  1.5 GiB   7 KiB   74 MiB  254 GiB  0.61  1.93
-                      host ocs-deviceset-1-data-0tllsb
  0       ssd  0.25000   1.00000  256 GiB  1.6 GiB  1.5 GiB   7 KiB   74 MiB  254 GiB  0.61
1.93  353      up                  osd.0
-51           0.25000         -  256 GiB   29 MiB  2.7 MiB   1 KiB   26 MiB  256 GiB  0.01
0.04    -                      host us-east-2-data-068z76
  5  us-east-2  0.25000   1.00000  256 GiB   29 MiB  2.7 MiB   1 KiB   26 MiB  256 GiB
0.01  0.04    0      up                  osd.5
-10           0.50000         -  512 GiB  1.6 GiB  1.5 GiB  12 KiB  118 MiB  510 GiB  0.31
0.99    -                  zone us-east-3
 -9           0.25000         -  256 GiB  1.6 GiB  1.5 GiB  11 KiB   92 MiB  254 GiB  0.61  1.95
-                      host ocs-deviceset-0-data-06fhxp
```

```
 1       ssd  0.25000   1.00000  256 GiB  1.6 GiB  1.5 GiB  11 KiB   92 MiB  254 GiB  0.61
1.95  353      up                  osd.1
-41             0.25000         -  256 GiB   29 MiB  2.7 MiB   1 KiB   26 MiB  256 GiB  0.01
0.04    -                   host us-east-3-data-04wvc8
 4  us-east-3  0.25000   1.00000  256 GiB   29 MiB  2.7 MiB   1 KiB   26 MiB  256 GiB
0.01  0.04    0      up                  osd.4
                       TOTAL  1.5 TiB  4.8 GiB  4.4 GiB  36 KiB  395 MiB  1.5 TiB  0.31
MIN/MAX VAR: 0.04/1.95  STDDEV: 0.29
```

In this example, OSD ID 3,5,4 are the replica-1 OSDs that need to be removed.

b. Force remove the identified replica-1 OSDs.
Perform the removal of the OSDs by following appropriate procedure applicable to your environment.

Repeat the following steps for all OSD Ids that you want to remove.

```
$ osd_id_to_remove=3
$ oc scale -n openshift-storage deployment rook-ceph-osd-${osd_id_to_remove} --
replicas=0
deployment.apps/rook-ceph-osd-3 scaled
$ oc process -n openshift-storage ocs-osd-removal -p
FAILED_OSD_IDS=${osd_id_to_remove} FORCE_OSD_REMOVAL=true |oc create -n
openshift-storage -f -
job.batch/ocs-osd-removal-job created
$ oc logs -l job-name=ocs-osd-removal-job -n openshift-storage --tail=-1 | egrep -i
'completed removal'
2025-06-20 10:33:13.153093 I | cephosd: completed removal of OSD 3
$ oc delete -n openshift-storage job ocs-osd-removal-job
job.batch "ocs-osd-removal-job" deleted
```

c. Check the OSD tree.

```
sh-5.1$ ceph osd df tree
ID   CLASS  WEIGHT   REWEIGHT  SIZE     RAW USE  DATA     OMAP     META
AVAIL    %USE  VAR   PGS  STATUS  TYPE NAME
 -1         0.75000         -  768 GiB  5.0 GiB  4.8 GiB  29 KiB  227 MiB  763 GiB  0.65  1.00
-          root default
 -5         0.75000         -  768 GiB  5.0 GiB  4.8 GiB  29 KiB  227 MiB  763 GiB  0.65  1.00
-            region us-east
-14         0.25000         -  256 GiB  1.7 GiB  1.6 GiB  11 KiB   63 MiB  254 GiB  0.65  0.99
-                zone us-east-1
-13         0.25000         -  256 GiB  1.7 GiB  1.6 GiB  11 KiB   63 MiB  254 GiB  0.65  0.99
-                    host ocs-deviceset-2-data-0kp6fl
  2    ssd  0.25000   1.00000  256 GiB  1.7 GiB  1.6 GiB  11 KiB   63 MiB  254 GiB  0.65
0.99  353      up                  osd.2
 -4         0.25000         -  256 GiB  1.7 GiB  1.6 GiB   7 KiB   80 MiB  254 GiB  0.65  1.00    -
zone us-east-2
 -3         0.25000         -  256 GiB  1.7 GiB  1.6 GiB   7 KiB   80 MiB  254 GiB  0.65  1.00    -
host ocs-deviceset-1-data-0tllsb
  0    ssd  0.25000   1.00000  256 GiB  1.7 GiB  1.6 GiB   7 KiB   80 MiB  254 GiB  0.65
1.00  353      up                  osd.0
-10         0.25000         -  256 GiB  1.7 GiB  1.6 GiB  11 KiB   84 MiB  254 GiB  0.65  1.00
-                zone us-east-3
 -9         0.25000         -  256 GiB  1.7 GiB  1.6 GiB  11 KiB   84 MiB  254 GiB  0.65  1.00
-                    host ocs-deviceset-0-data-06fhxp
```

```
 1    ssd  0.25000   1.00000  256 GiB  1.7 GiB  1.6 GiB  11 KiB   84 MiB  254 GiB  0.65
1.00  353      up                  osd.1
                      TOTAL  768 GiB  5.0 GiB  4.8 GiB  31 KiB  227 MiB  763 GiB  0.65
MIN/MAX VAR: 0.99/1.00  STDDEV: 0.00
```

2.2.1. Recovering after OSD lost from single replica

When using replica 1, a storage class with a single replica, data loss is guaranteed when an OSD is lost. Lost OSDs go into a failing state. Use the following steps to recover after OSD loss.

Procedure

1. Follow these recovery steps to get your applications running again after data loss from replica 1. You first need to identify the domain where the failing OSD is.

2. If you know which failure domain the failing OSD is in, run the following command to get the exact **replica1-pool-name** required for the next steps. If you do not know where the failing OSD is, skip to step 2.

   ```
   $ oc get cephblockpools
   ```

   Example output:

   ```
   NAME                                 PHASE
   ocs-storagecluster-cephblockpool             Ready
   ocs-storagecluster-cephblockpool-us-south-1   Ready
   ocs-storagecluster-cephblockpool-us-south-2   Ready
   ocs-storagecluster-cephblockpool-us-south-3   Ready
   ```

   Copy the corresponding failure domain name for use in next steps, then skip to step 4.

3. Find the OSD pod that is in **Error** state or **CrashLoopBackoff** state to find the failing OSD:

   ```
   $ oc get pods -n openshift-storage -l app=rook-ceph-osd  | grep 'CrashLoopBackOff\|Error'
   ```

4. Identify the replica-1 pool that had the failed OSD.

   a. Identify the node where the failed OSD was running:

      ```
      failed_osd_id=0 #replace with the ID of the failed OSD
      ```

   b. Identify the failureDomainLabel for the node where the failed OSD was running:

      ```
      failure_domain_label=$(oc get storageclass ocs-storagecluster-ceph-non-resilient-rbd -o
      yaml | grep domainLabel |head -1 |awk -F':' '{print $2}')
      ```

      ```
      failure_domain_value=$”(oc get pods $failed_osd_id -oyaml |grep topology-location-zone
      |awk ‘{print $2}’)”
      ```

      The output shows the replica-1 pool name whose OSD is failing, for example:

      ```
      replica1-pool-name= "ocs-storagecluster-cephblockpool-$failure_domain_value”
      ```

where **$failure_domain_value** is the failureDomainName.

5. Delete the replica-1 pool.

   a. Connect to the toolbox pod:

   ```
   toolbox=$(oc get pod -l app=rook-ceph-tools -n openshift-storage -o
   jsonpath='{.items[*].metadata.name}')

   oc rsh $toolbox -n openshift-storage
   ```

   b. Delete the replica-1 pool. Note that you have to enter the replica-1 pool name twice in the command, for example:

   ```
   ceph osd pool rm <replica1-pool-name> <replica1-pool-name> --yes-i-really-really-
   mean-it
   ```

   Replace **replica1-pool-name** with the failure domain name identified earlier.

6. Purge the failing OSD by following the steps in section "Replacing operational or failed storage devices" based on your platform in the Replacing devices guide.

7. Restart the rook-ceph operator:

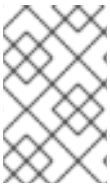
   ```
   $ oc delete pod -l rook-ceph-operator -n openshift-storage
   ```

8. Recreate any affected applications in that avaialbity zone to start using the new pool with same name.

CHAPTER 3. PERSISTENT VOLUME ENCRYPTION

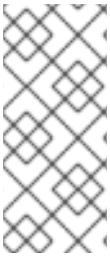
Persistent volume (PV) encryption guarantees isolation and confidentiality between tenants (applications). Before you can use PV encryption, you must create a storage class for PV encryption. Persistent volume encryption is only available for RBD PVs.

OpenShift Data Foundation supports storing encryption passphrases in HashiCorp Vault and Thales CipherTrust Manager. You can create an encryption enabled storage class using an external key management system (KMS) for persistent volume encryption. You need to configure access to the KMS before creating the storage class.



NOTE

For PV encryption, you must have a valid Red Hat OpenShift Data Foundation Advanced subscription. For more information, see the OpenShift Data Foundation subscriptions knowledge base article.



NOTE

When you create an RBD PVC with encryption enabled, the usable capacity of the volume is slightly less than the requested size. This reduction occurs because a portion of the volume's space is reserved for encryption metadata and overhead required by the encryption layer. As a result, the total available storage within the encrypted volume may appear smaller than the requested PVC size.

Additional resources

- OpenShift Data Foundation subscriptions knowledge base article

3.1. ACCESS CONFIGURATION FOR KEY MANAGEMENT SYSTEM (KMS)

Based on your use case, you need to configure access to KMS using one of the following ways:

- Using **vaulttokens**: allows users to authenticate using a token
- Using **Thales CipherTrust Manager**: uses Key Management Interoperability Protocol (KMIP)
- Using **vaulttenantsa**: allows users to use **serviceaccounts** to authenticate with **Vault**

3.1.1. Configuring access to KMS using vaulttokens

Use this procedure to configure access to Key Management System (KMS) using **vaulttokens** for vault authentication.

Prerequisites

- The OpenShift Data Foundation cluster is in **Ready** state.
- On the external key management system (KMS),
  - Ensure that a policy with a token exists and the key value backend path in **Vault** is enabled.

- Ensure that you are using signed certificates on your **Vault** servers.

Procedure

1. Create a secret in the tenant's namespace using the OpenShift Container Platform web console:
   a. Navigate to **Workloads → Secrets**
2. Click **Create → Key/value secret**
3. Enter **Secret Name** as **ceph-csi-kms-token**.
4. Enter **Key** as **token**.
5. Enter **Value**.
   It is the token from Vault. You can either click **Browse** to select and upload the file containing the token or enter the token directly in the text box.
6. Click **Create**.



NOTE

The token can be deleted only after all the encrypted PVCs using the **ceph-csi-kms-token** have been deleted.

3.1.2. Configuring access to KMS using Thales CipherTrust Manager

Use this procedure to configure access to Key Management System (KMS) using Thales CipherTrust Manager. .Prerequisites

1. Create a KMIP client if one does not exist. From the user interface, select **KMIP → Client Profile → Add Profile**.
   a. Add the **CipherTrust** username to the **Common Name** field during profile creation.
2. Create a token be navigating to **KMIP → Registration Token → New Registration Token**. Copy the token for the next step.
3. To register the client, navigate to **KMIP → Registered Clients → Add Client**. Specify the **Name**. Paste the **Registration Token** from the previous step, then click **Save**.
4. Download the Private Key and Client Certificate by clicking **Save Private Key** and **Save Certificate** respectively.
5. To create a new KMIP interface, navigate to **Admin Settings → Interfaces → Add Interface**.
   a. Select **KMIP Key Management Interoperability Protocol** and click **Next**.
   b. Select a free **Port**.
   c. Select **Network Interface** as **all**.
   d. Select **Interface Mode** as **TLS, verify client cert, user name taken from client cert, auth request is optional**.

e. (Optional) You can enable hard delete to delete both meta-data and material when the key is deleted. It is disabled by default.

f. Select the CA to be used, and click **Save**.

6. To get the server CA certificate, click on the Action menu **(⋮)** on the right of the newly created interface, and click **Download Certificate**.

Procedure

1. To create a key to act as the Key Encryption Key (KEK) for storageclass encryption, follow the steps below:

   a. Navigate to **Keys → Add Key**.

   b. Enter **Key Name**.

   c. Set the **Algorithm** and **Size** to **AES** and **256** respectively.

   d. Enable **Create a key in Pre-Active state**and set the date and time for activation.

   e. Ensure that **Encrypt** and **Decrypt** are enabled under **Key Usage**.

   f. Copy the ID of the newly created Key to be used as the Unique Identifier during deployment.

3.1.3. Configuring access to KMS using vaulttenantsa

Prerequisites

- The OpenShift Data Foundation cluster is in **Ready** state.
- On the external key management system (KMS),
  - Ensure that a policy exists and the key value backend path in Vault is enabled.
  - Ensure that you are using signed certificates on your Vault servers.
- Create the following serviceaccount in the tenant namespace as shown below:

```
$ cat <<EOF | oc create -f -
apiVersion: v1
kind: ServiceAccount
metadata:
   name: ceph-csi-vault-sa
EOF
```

Procedure

1. Configure the Kubernetes authentication method before OpenShift Data Foundation can authenticate with and start using **Vault**. The following instructions create and configure **serviceAccount**, **ClusterRole**, and **ClusterRoleBinding** required to allow OpenShift Data Foundation to authenticate with **Vault**.
   Apply the following YAML to your Openshift cluster:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: rbd-csi-vault-token-review
---
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: rbd-csi-vault-token-review
rules:
  - apiGroups: ["authentication.k8s.io"]
    resources: ["tokenreviews"]
    verbs: ["create", "get", "list"]

---
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: rbd-csi-vault-token-review
subjects:
  - kind: ServiceAccount
    name: rbd-csi-vault-token-review
    namespace: openshift-storage
roleRef:
  kind: ClusterRole
  name: rbd-csi-vault-token-review
  apiGroup: rbac.authorization.k8s.io
```

2. Create a secret for serviceaccount token and CA certificate.

```
$ cat <<EOF | oc create -f -
apiVersion: v1
kind: Secret
metadata:
  name: rbd-csi-vault-token-review-token
  namespace: openshift-storage
  annotations:
    kubernetes.io/service-account.name: "rbd-csi-vault-token-review"
type: kubernetes.io/service-account-token
data: {}
EOF
```

3. Get the token and the CA certificate from the secret.

```
$ SA_JWT_TOKEN=$(oc -n openshift-storage get secret rbd-csi-vault-token-review-token -o
jsonpath="{.data['token']}" | base64 --decode; echo)
$ SA_CA_CRT=$(oc -n openshift-storage get secret rbd-csi-vault-token-review-token -o
jsonpath="{.data['ca\.crt']}" | base64 --decode; echo)
```

4. Retrieve the OpenShift cluster endpoint.

```
$ OCP_HOST=$(oc config view --minify --flatten -o jsonpath="{.clusters[0].cluster.server}")
```

5. Use the information collected in the previous steps to set up the kubernetes authentication method in Vault as shown:

```
$ vault auth enable kubernetes
$ vault write auth/kubernetes/config \
        token_reviewer_jwt="$SA_JWT_TOKEN" \
        kubernetes_host="$OCP_HOST" \
        kubernetes_ca_cert="$SA_CA_CRT"
```

6. Create a role in Vault for the tenant namespace:

```
$ vault write "auth/kubernetes/role/csi-kubernetes" bound_service_account_names="ceph-csi-vault-sa" bound_service_account_namespaces=<tenant_namespace> policies=<policy_name_in_vault>
```

csi-kubernetes is the default role name that OpenShift Data Foundation looks for in Vault. The default service account name in the tenant namespace in the OpenShift Data Foundation cluster is **ceph-csi-vault-sa**. These default values can be overridden by creating a ConfigMap in the tenant namespace.

For more information about overriding the default names, see Overriding Vault connection details using tenant ConfigMap.

To associate the role to the service account **ceph-csi-vault-sa** for all namespaces instead of a single namespace, use the following command:

```
$ oc adm policy add-role-to-user system:auth-delegator system:serviceaccount:*:ceph-csi-vault-sa
```

7. **Sample YAML**
To create a storageclass that uses the **vaulttenantsa** method for PV encryption, you must either edit the existing ConfigMap or create a ConfigMap named **csi-kms-connection-details** that will hold all the information needed to establish the connection with Vault.

The sample yaml given below can be used to update or create the **csi-kms-connection-detail** ConfigMap:

```
apiVersion: v1
data:
  vault-tenant-sa: |-
    {
      "encryptionKMSType": "vaulttenantsa",
      "vaultAddress": "<https://hostname_or_ip_of_vault_server:port>",
      "vaultTLSServerName": "<vault TLS server name>",
      "vaultAuthPath": "/v1/auth/kubernetes/login",
      "vaultAuthNamespace": "<vault auth namespace name>"
      "vaultNamespace": "<vault namespace name>",
      "vaultBackendPath": "<vault backend path name>",
      "vaultCAFromSecret": "<secret containing CA cert>",
      "vaultClientCertFromSecret": "<secret containing client cert>",
      "vaultClientCertKeyFromSecret": "<secret containing client private key>",
      "tenantSAName": "<service account name in the tenant namespace>"
    }
metadata:
  name: csi-kms-connection-details
```

encryptionKMSType	Set to **vaulttenantsa** to use service accounts for authentication with vault.
vaultAddress	The hostname or IP address of the vault server with the port number.
vaultTLSServerName	(Optional) The vault TLS server name
vaultAuthPath	(Optional) The path where kubernetes auth method is enabled in Vault. The default path is **kubernetes**. If the auth method is enabled in a different path other than **kubernetes**, this variable needs to be set as**"/v1/auth/<path>/login"**.
vaultAuthNamespace	(Optional) The Vault namespace where kubernetes auth method is enabled.
vaultNamespace	(Optional) The Vault namespace where the backend path being used to store the keys exists
vaultBackendPath	The backend path in Vault where the encryption keys will be stored
vaultCAFromSecret	The secret in the OpenShift Data Foundation cluster containing the CA certificate from Vault
vaultClientCertFromSecret	The secret in the OpenShift Data Foundation cluster containing the client certificate from Vault
vaultClientCertKeyFromSecret	The secret in the OpenShift Data Foundation cluster containing the client private key from Vault
tenantSAName	(Optional) The service account name in the tenant namespace. The default value is **ceph-csi-vault-sa**. If a different name is to be used, this variable has to be set accordingly.

3.2. CREATING A STORAGE CLASS FOR PERSISTENT VOLUME ENCRYPTION

Prerequisites

Based on your use case, you must ensure to configure access to KMS for one of the following:

- Using **vaulttokens**: Ensure to configure access as described in Configuring access to KMS using **vaulttokens**
- Using **vaulttenantsa**: Ensure to configure access as described in Configuring access to KMS using **vaulttenantsa**
- Using Thales CipherTrust Manager (using KMIP): Ensure to configure access as described in Configuring access to KMS using Thales CipherTrust Manager
- (For users on Azure platform only) Using Azure Vault: Ensure to set up client authentication and fetch the client credentials from Azure using the following steps:
    1. Create Azure Vault. For more information, see Quickstart: Create a key vault using the Azure portal in Microsoft product documentation.
    2. Create Service Principal with certificate based authentication. For more information, see Create an Azure service principal with Azure CLI in Microsoft product documentation.
    3. Set Azure Key Vault role based access control (RBAC). For more information, see Enable Azure RBAC permissions on Key Vault.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → StorageClasses**.
2. Click **Create Storage Class**
3. Enter the storage class **Name** and **Description**.
4. Select either **Delete** or **Retain** for the **Reclaim Policy**. By default, **Delete** is selected.
5. Select either **Immediate** or **WaitForFirstConsumer** as the **Volume binding mode**. **WaitForConsumer** is set as the default option.
6. Select **RBD Provisioner openshift-storage.rbd.csi.ceph.com** which is the plugin used for provisioning the persistent volumes.
7. Select **Storage Pool** where the volume data is stored from the list or create a new pool.
8. Select the **Enable encryption** checkbox.
    - Choose one of the following options to set the KMS connection details:
        - **Choose existing KMS connection**: Select an existing KMS connection from the drop-down list. The list is populated from the the connection details available in the **csi-kms-connection-details** ConfigMap.
            a. Select the **Provider** from the drop down.
            b. Select the **Key service** for the given provider from the list.
        - **Create new KMS connection**: This is applicable for **vaulttokens** and **Thales CipherTrust Manager (using KMIP)** only.
            a. Select one of the following **Key Management Service Provider** and provide the required details.
                - **Vault**

i. Enter a unique **Connection Name**, host **Address** of the Vault server ('https://<hostname or ip>'), Port number and **Token**.

ii. Expand **Advanced Settings** to enter additional settings and certificate details based on your **Vault** configuration:

A. Enter the Key Value secret path in **Backend Path** that is dedicated and unique to OpenShift Data Foundation.

B. Optional: Enter **TLS Server Name** and **Vault Enterprise Namespace**.

C. Upload the respective PEM encoded certificate file to provide the **CA Certificate**, **Client Certificate** and **Client Private Key** .

D. Click **Save**.

- **Thales CipherTrust Manager (using KMIP)**

i. Enter a unique **Connection Name**.

ii. In the **Address** and **Port** sections, enter the IP of Thales CipherTrust Manager and the port where the KMIP interface is enabled. For example, **Address**: 123.34.3.2, **Port**: 5696.

iii. Upload the **Client Certificate**, **CA certificate**, and **Client Private Key**.

iv. Enter the **Unique Identifier** for the key to be used for encryption and decryption, generated above.

v. The **TLS Server** field is optional and used when there is no DNS entry for the KMIP endpoint. For example, **kmip_all_<port>.ciphertrustmanager.local**.

- **Azure Key Vault** (Only for Azure users on Azure platform)
For information about setting up client authentication and fetching the client credentials, see the Prerequisites in Creating an OpenShift Data Foundation cluster section of the *Deploying OpenShift Data Foundation using Microsoft Azure* guide.

i. Enter a unique **Connection name** for the key management service within the project.

ii. Enter **Azure Vault URL**.

iii. Enter **Client ID**.

iv. Enter **Tenant ID**.

v. Upload **Certificate** file in **.PEM** format and the certificate file must include a client certificate and a private key.

b. Click **Save**.

c. Click **Create**.

9. Edit the ConfigMap to add the **vaultBackend** parameter if the HashiCorp Vault setup does not allow automatic detection of the Key/Value (KV) secret engine API version used by the backend path.



NOTE

vaultBackend is an optional parameters that is added to the **configmap** to specify the version of the KV secret engine API associated with the backend path. Ensure that the value matches the KV secret engine API version that is set for the backend path, otherwise it might result in a failure during persistent volume claim (PVC) creation.

a. Identify the encryptionKMSID being used by the newly created storage class.

i. On the OpenShift Web Console, navigate to **Storage → Storage Classes**.

ii. Click the **Storage class** name → **YAML** tab.

iii. Capture the **encryptionKMSID** being used by the storage class.
Example:

```
encryptionKMSID: 1-vault
```

b. On the OpenShift Web Console, navigate to **Workloads → ConfigMaps**.

c. To view the KMS connection details, click **csi-kms-connection-details**.

d. Edit the ConfigMap.

i. Click Action menu **(⋮) → Edit ConfigMap**.

ii. Add the **vaultBackend** parameter depending on the backend that is configured for the previously identified **encryptionKMSID**.
You can assign **kv** for KV secret engine API, version 1 and **kv-v2** for KV secret engine API, version 2.

Example:

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: csi-kms-connection-details
[...]
data:
  1-vault: |-
    {
      "encryptionKMSType": "vaulttokens",
      "kmsServiceName": "1-vault",
      [...]
      "vaultBackend": "kv-v2"
    }
  2-vault: |-
    {
      "encryptionKMSType": "vaulttenantsa",
```

```
  [...]
  "vaultBackend": "kv"
}
```

iii. Click Save

Next steps

- The storage class can be used to create encrypted persistent volumes. For more information, see managing persistent volume claims.



IMPORTANT

Red Hat works with the technology partners to provide this documentation as a service to the customers. However, Red Hat does not provide support for the HashiCorp product. For technical assistance with this product, contact HashiCorp.

3.2.1. Overriding Vault connection details using tenant ConfigMap

The Vault connections details can be reconfigured per tenant by creating a ConfigMap in the Openshift namespace with configuration options that differ from the values set in the **csi-kms-connection-details** ConfigMap in the **openshift-storage** namespace. The ConfigMap needs to be located in the tenant namespace. The values in the ConfigMap in the tenant namespace will override the values set in the **csi-kms-connection-details** ConfigMap for the encrypted Persistent Volumes created in that namespace.

Procedure

1. Ensure that you are in the tenant namespace.
2. Click on **Workloads → ConfigMaps**.
3. Click on **Create ConfigMap**.
4. The following is a sample yaml. The values to be overidden for the given tenant namespace can be specified under the **data** section as shown below:

```
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: ceph-csi-kms-config
data:
  vaultAddress: "<vault_address:port>"
  vaultBackendPath: "<backend_path>"
  vaultTLSServerName: "<vault_tls_server_name>"
  vaultNamespace: "<vault_namespace>"
```

5. After the yaml is edited, click on **Create**.

3.3. ENABLING AND DISABLING KEY ROTATION WHEN USING KMS

Security common practices require periodic encryption of key rotation. You can enable or disable key rotation when using KMS.

3.3.1. Enforcing precedence for key rotation

Configure key rotation precedence to control whether the rotation schedule is read from the PVC annotation or the storage class in OpenShift Data Foundation.

In key rotation, precedence refers to the order in which the system checks for scheduled annotations. In OpenShift Data Foundation, the default precedence is set to storage class (recommended). This means the system reads annotations only from the storage class.

However, if you want the system to check the persistent volume claim (PVC) first and then fall back to the storage class, you can configure this behavior by setting the **schedule-precedence** to PVC in the CSI-addons ConfigMap.

Procedure

1. Define the **ConfigMap** for csi-addons to set PVC precedence:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: csi-addons-config
  namespace: openshift-storage
data:
  "schedule-precedence": "pvc"
```

2. Restart the **csi-addons** operator pod for the changes to take effect:

```
oc delete po -n openshift-storage -l "app.kubernetes.io/name=csi-addons"
```

3.3.2. Enabling key rotation

Enable key rotation to automatically re-encrypt persistent volume data at a scheduled interval using the CSI Addons framework in OpenShift Data Foundation.

To enable key rotation, add the annotation **keyrotation.csiaddons.openshift.io/schedule: <value>** to a **PersistentVolumeClaim**, **Namespace**, or **StorageClass** (in decreasing order of precedence).



IMPORTANT

Key rotation is only supported for RBD backed volumes.

3.3.2.1. Annotating a namespace for key rotation

Add the key rotation annotation to a namespace so that all RBD-backed persistent volumes in that namespace are re-encrypted on the configured schedule.

Prerequisites

- Key rotation is only supported for RBD backed volumes.

Procedure

Procedure

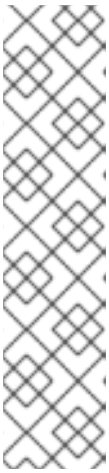
1. Verify the namespace exists:

```
$ oc get namespace default
NAME      STATUS   AGE
default   Active   5d2h
```

2. Annotate the namespace with the desired rotation schedule:

```
$ oc annotate namespace default "keyrotation.csiaddons.openshift.io/schedule=@weekly"
namespace/default annotated
```

<value> can be **@hourly**, **@daily**, **@weekly**, **@monthly**, or **@yearly**. If **<value>** is empty, the default is **@weekly**.



NOTE

- When any schedule for key rotation is used, including **@daily**, **@weekly**, **@monthly**, and cron expressions such as **0 * * * ***, multiple CronJobs are created. To prevent all Jobs from running simultaneously and overloading high-scale systems, executions are staggered within a 2-hour window from the scheduled start time (e.g., **@weekly** would be 00:00, 00:05, 00:10, and so on).
    - If the configured schedule interval is less than 2 hours, the stagger window is reduced to match the interval. For example, a job running every 30 minutes will be staggered within a 00:00–00:30 window rather than 00:00–02:00.

3.3.2.2. Annotating a storage class for key rotation

Add the key rotation annotation to a storage class so that all RBD-backed persistent volumes provisioned from that storage class are re-encrypted on the configured schedule.

Prerequisites

- Key rotation is only supported for RBD backed volumes.

Procedure

1. Verify the storage class exists:

```
$ oc get storageclass rbd-sc
NAME      PROVISIONER        RECLAIMPOLICY   VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION   AGE
rbd-sc    rbd.csi.ceph.com   Delete          Immediate          true                5d2h
```

2. Annotate the storage class with the desired rotation schedule:

```
$ oc annotate storageclass rbd-sc "keyrotation.csiaddons.openshift.io/schedule=@weekly"
storageclass.storage.k8s.io/rbd-sc annotated
```

<value> can be **@hourly**, **@daily**, **@weekly**, **@monthly**, or **@yearly**. If **<value>** is empty, the default is **@weekly**.

3.3.2.3. Annotating a PersistentVolumeClaim for key rotation

Add the key rotation annotation directly to a PersistentVolumeClaim to re-encrypt that volume on the configured schedule, overriding any namespace or storage class annotation.

Prerequisites

- Key rotation is only supported for RBD backed volumes.

Procedure

1. Verify the PersistentVolumeClaim exists:

```
$ oc get pvc data-pvc
NAME      STATUS   VOLUME                                     CAPACITY   ACCESS MODES
STORAGECLASS      AGE
data-pvc  Bound    pvc-f37b8582-4b04-4676-88dd-e1b95c6abf74   1Gi        RWO
default           20h
```

2. Annotate the PVC with the desired rotation schedule:

```
$ oc annotate pvc data-pvc "keyrotation.csiaddons.openshift.io/schedule=@weekly"
persistentvolumeclaim/data-pvc annotated
```

<value> can be **@hourly**, **@daily**, **@weekly**, **@monthly**, or **@yearly**. If **<value>** is empty, the default is **@weekly**.

3. Verify the encryption key rotation cron job was created:

```
$ oc get encryptionkeyrotationcronjobs.csiaddons.openshift.io
NAME                  SCHEDULE    SUSPEND   ACTIVE   LASTSCHEDULE   AGE
data-pvc-1642663516   @weekly                                       3s
```

4. Optional: Update the schedule by overwriting the annotation:

```
$ oc annotate pvc data-pvc "keyrotation.csiaddons.openshift.io/schedule=*/1 * * * *" --
overwrite=true
persistentvolumeclaim/data-pvc annotated
```

5. Verify the updated cron job schedule:

```
$ oc get encryptionkeyrotationcronjobs.csiaddons.openshift.io
NAME                  SCHEDULE    SUSPEND   ACTIVE   LASTSCHEDULE   AGE
data-pvc-1642664617   */1 * * * *                                   3s
```

3.3.3. Disabling key rotation

Disable automatic key rotation for all PVCs in a storage class or for a specific persistent volume claim in OpenShift Data Foundation.

You can disable key rotation for the following:

- All the persistent volume claims (PVCs) of storage class
- A specific PVC

Disabling key rotation for all PVCs of a storage class

To disable key rotation for all PVCs, update the annotation of the storage class:

```
$ oc get storageclass rbd-sc
NAME      PROVISIONER        RECLAIMPOLICY   VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION   AGE
rbd-sc    rbd.csi.ceph.com   Delete          Immediate           true                   5d2h
```

```
$ oc annotate storageclass rbd-sc "keyrotation.csiaddons.openshift.io/enable: false"
storageclass.storage.k8s.io/rbd-sc annotated
```

Disabling key rotation for a specific persistent volume claim

Procedure

1. Identify the **EncryptionKeyRotationCronJob** CR for the PVC you want to disable key rotation on:

   ```
   $ oc get encryptionkeyrotationcronjob -o jsonpath='{range .items[?
   (@.spec.jobTemplate.spec.target.persistentVolumeClaim=="<PVC_NAME>")]}
   {.metadata.name}{"\n"}{end}'
   ```

   Where **<PVC_NAME>** is the name of the PVC that you want to disable.

2. Apply the following to the **EncryptionKeyRotationCronJob** CR from the previous step to disable the key rotation:

   a. Update the **csiaddons.openshift.io/state** annotation from **managed** to **unmanaged**:

   ```
   $ oc annotate encryptionkeyrotationcronjob <encryptionkeyrotationcronjob_name>
   "csiaddons.openshift.io/state=unmanaged" --overwrite=true
   ```

   Where **<encryptionkeyrotationcronjob_name>** is the name of the **EncryptionKeyRotationCronJob** CR.

   b. Add **suspend: true** under the **spec** field:

   ```
   $ oc patch encryptionkeyrotationcronjob <encryptionkeyrotationcronjob_name> -p
   '{"spec": {"suspend": true}}' --type=merge.
   ```

   The key rotation will be disabled for the PVC.

CHAPTER 4. ENABLING AND DISABLING ENCRYPTION IN-TRANSIT POST DEPLOYMENT

You can enable encryption in-transit for the existing clusters after the deployment of clusters both in internal and external modes.

4.1. ENABLING ENCRYPTION IN-TRANSIT AFTER DEPLOYMENT IN INTERNAL MODE

Enable encryption in-transit for OpenShift Data Foundation deployed in internal mode to encrypt all data moving between Ceph components over the network.

Prerequisites

- OpenShift Data Foundation is deployed and a storage cluster is created.

Procedure

1. Patch the storagecluster to add encryption **enabled** as true to the storage cluster spec:

```
$ oc patch storagecluster ocs-storagecluster -n openshift-storage --type json --patch  '[{ "op": "replace", "path": "/spec/network", "value": {"connections": {"encryption": {"enabled": true}}} }]'
storagecluster.ocs.openshift.io/ocs-storagecluster patched
```

2. Check the configurations.

```
$ oc get storagecluster ocs-storagecluster -n openshift-storage -o yaml | yq '.spec.network'
connections:
  encryption:
    enabled: true
```

3. Wait for around 10 minutes for ceph daemons to restart and then check the pods.

```
$ oc get pods -n openshift-storage | grep rook-ceph
rook-ceph-crashcollector-ip-10-0-2-111.ec2.internal-796ffcm9kn9   1/1     Running     0          5m11s
rook-ceph-crashcollector-ip-10-0-27-61.ec2.internal-854b4d8sk5z   1/1     Running     0          5m9s
rook-ceph-crashcollector-ip-10-0-33-53.ec2.internal-589d9f4f8vx   1/1     Running     0          5m7s
rook-ceph-exporter-ip-10-0-2-111.ec2.internal-6d48cdc5fd-2tmsl    1/1     Running     0          5m9s
rook-ceph-exporter-ip-10-0-27-61.ec2.internal-546c66c7cc-9lnpz    1/1     Running     0          5m7s
rook-ceph-exporter-ip-10-0-33-53.ec2.internal-b5555994c-x8mzz     1/1     Running     0          5m5s
rook-ceph-mds-ocs-storagecluster-cephfilesystem-a-7bd754f6vwps2   2/2     Running     0          4m56s
rook-ceph-mds-ocs-storagecluster-cephfilesystem-b-6cc5cc647c78m   2/2     Running     0          4m30s
rook-ceph-mgr-a-6f8467578d-f8279                                  3/3     Running     0          3m40s
rook-ceph-mgr-b-66754d99cf-9q58g                                  3/3     Running     0          3m27s
rook-ceph-mon-a-75bc5dd655-tvdqf                                  2/2     Running     0          4m7s
```

```
rook-ceph-mon-b-6b6d4d9b4c-tjbpz                                  2/2     Running     0          4m55s
rook-ceph-mon-c-7456bb5f67-rtwpj                                  2/2     Running     0          4m32s
rook-ceph-operator-7b5b9cdb9b-tvmb6                               1/1     Running     0          45m
rook-ceph-osd-0-b78dd99f6-n4wbm                                   2/2     Running     0          3m3s
rook-ceph-osd-1-5887bf6d8d-2sncc                                  2/2     Running     0          2m39s
rook-ceph-osd-2-784b59c4c8-44phh                                  2/2     Running     0          2m14s
rook-ceph-osd-prepare-a075cf185c9b2e5d92ec3f7769565e38-ztrms      0/1     Completed   0
42m
rook-ceph-osd-prepare-b4b48dc5e3bef99ab377e2a255a9142a-mvgnd      0/1     Completed
0          42m
rook-ceph-osd-prepare-fae2ea2ad4aacbf62010ae5b60b87f57-6t9l5      0/1     Completed   0
42m
```

```
$ oc get storagecluster -n openshift-storage
NAME                 AGE   PHASE   EXTERNAL   CREATED AT            VERSION
ocs-storagecluster   27m   Ready              2024-11-06T16:15:26Z   4.22.0
```

4. Remount existing volumes.
   Depending on your best practices for application maintenance, you can choose the best approach for your environment to remount or remap volumes. One way to remount is to delete the existing application pod and bring up another application pod to use the volume. Another option is to drain the nodes where the applications are running. This ensures that the volume is unmounted from the current pod and then mounted to a new pod, allowing for remapping or remounting of the volume.

4.2. DISABLING ENCRYPTION IN-TRANSIT AFTER DEPLOYMENT IN INTERNAL MODE

Disable encryption in-transit for OpenShift Data Foundation deployed in internal mode by modifying the storage cluster custom resource.

Prerequisites

- OpenShift Data Foundation is deployed and a storage cluster is created.
- Encryption in-transit is enabled.

Procedure

1. Patch the storagecluster to update encryption **enabled** as **false** in the storage cluster spec:

```
~ $ oc patch storagecluster ocs-storagecluster -n openshift-storage --type json --patch  '[{
"op": "replace", "path": "/spec/network", "value": {"connections": {"encryption": {"enabled":
false}}} }]'
storagecluster.ocs.openshift.io/ocs-storagecluster patched
```

2. Check the configurations.

```
$ oc get storagecluster ocs-storagecluster -n openshift-storage -o yaml | yq '.spec.network'

connections:
  encryption:
    enabled: false
```

3. Wait for around 10 minutes for ceph daemons to restart and then check the pods.

```
$ oc get pods -n openshift-storage | grep rook-ceph
rook-ceph-crashcollector-ip-10-0-2-111.ec2.internal-796ffcm9kn9   1/1     Running     0
5m11s
rook-ceph-crashcollector-ip-10-0-27-61.ec2.internal-854b4d8sk5z   1/1     Running     0
5m9s
rook-ceph-crashcollector-ip-10-0-33-53.ec2.internal-589d9f4f8vx   1/1     Running     0
5m7s
rook-ceph-exporter-ip-10-0-2-111.ec2.internal-6d48cdc5fd-2tmsl    1/1     Running     0
5m9s
rook-ceph-exporter-ip-10-0-27-61.ec2.internal-546c66c7cc-9lnpz    1/1     Running     0
5m7s
rook-ceph-exporter-ip-10-0-33-53.ec2.internal-b5555994c-x8mzz     1/1     Running     0
5m5s
rook-ceph-mds-ocs-storagecluster-cephfilesystem-a-7bd754f6vwps2   2/2     Running     0
4m56s
rook-ceph-mds-ocs-storagecluster-cephfilesystem-b-6cc5cc647c78m   2/2     Running     0
4m30s
rook-ceph-mgr-a-6f8467578d-f8279                                  3/3     Running     0          3m40s
rook-ceph-mgr-b-66754d99cf-9q58g                                  3/3     Running     0          3m27s
rook-ceph-mon-a-75bc5dd655-tvdqf                                  2/2     Running     0          4m7s
rook-ceph-mon-b-6b6d4d9b4c-tjbpz                                  2/2     Running     0          4m55s
rook-ceph-mon-c-7456bb5f67-rtwpj                                  2/2     Running     0          4m32s
rook-ceph-operator-7b5b9cdb9b-tvmb6                               1/1     Running     0          45m
rook-ceph-osd-0-b78dd99f6-n4wbm                                   2/2     Running     0          3m3s
rook-ceph-osd-1-5887bf6d8d-2sncc                                  2/2     Running     0          2m39s
rook-ceph-osd-2-784b59c4c8-44phh                                  2/2     Running     0          2m14s
rook-ceph-osd-prepare-a075cf185c9b2e5d92ec3f7769565e38-ztrms      0/1     Completed   0
42m
rook-ceph-osd-prepare-b4b48dc5e3bef99ab377e2a255a9142a-mvgnd      0/1     Completed
0          42m
rook-ceph-osd-prepare-fae2ea2ad4aacbf62010ae5b60b87f57-6t9l5      0/1     Completed   0
42m
```

```
$ oc get storagecluster -n openshift-storage
NAME                 AGE   PHASE   EXTERNAL   CREATED AT             VERSION
ocs-storagecluster   27m   Ready              2024-11-06T16:15:26Z   4.22.0
```

4. Remount existing volumes.
   Depending on your best practices for application maintenance, you can choose the best approach for your environment to remount or remap volumes. One way to remount is to delete the existing application pod and bring up another application pod to use the volume. Another option is to drain the nodes where the applications are running. This ensures that the volume is unmounted from the current pod and then mounted to a new pod, allowing for remapping or remounting of the volume.

4.3. ENABLING ENCRYPTION IN-TRANSIT AFTER DEPLOYMENT IN EXTERNAL MODE

Enable encryption in-transit for OpenShift Data Foundation deployed in external mode to encrypt all data moving between Ceph components over the network.

Prerequisites

- OpenShift Data Foundation is deployed and a storage cluster is created.

Procedure

1. Patch the storagecluster to add encryption **enabled** as true the storage cluster spec:

```
$ oc patch storagecluster ocs-external-storagecluster -n openshift-storage --type json --patch
'[{ "op": "replace", "path": "/spec/network", "value": {"connections": {"encryption": {"enabled":
true}}} }]'
storagecluster.ocs.openshift.io/ocs-external-storagecluster patched
```

2. Check the connection settings in the CR.

```
oc get storagecluster
NAME                          AGE   PHASE   EXTERNAL   CREATED AT            VERSION
ocs-external-storagecluster   9h    Ready   true       2024-11-06T20:48:03Z   4.22.0
```

```
$ oc get storagecluster ocs-external-storagecluster -o yaml | yq '.spec.network.connections'
encryption:
  enabled: true
```

3. Apply encryption in-transit settings on the Red Hat Ceph Storage cluster.

```
root@ceph-client ~]# ceph config set global ms_client_mode secure
[root@ceph-client ~]# ceph config set global ms_cluster_mode secure
[root@ceph-client ~]# ceph config set global ms_service_mode secure
[root@ceph-client ~]# ceph config set global rbd_default_map_options ms_mode=secure
```

4. Check the settings.

```
[root@ceph-client ~]# ceph config dump | grep ms_
ceph config dump | grep ms_
global   basic      ms_client_mode secure                  *
global   basic      ms_cluster_mode secure                 *
global   basic      ms_service_mode secure                 *
global   advanced   rbd_default_map_options ms_mode=secure   *
```

5. Restart all Ceph daemons.

```
[root@ceph-client ~]# ceph orch ls --format plain | tail -n +2 | awk '{print $1}' | xargs -I {} ceph
orch restart {}
Scheduled to restart alertmanager.osd-0 on host 'osd-0'
Scheduled to restart ceph-exporter.osd-0 on host 'osd-0'
Scheduled to restart ceph-exporter.osd-2 on host 'osd-2'
Scheduled to restart ceph-exporter.osd-3 on host 'osd-3'
Scheduled to restart ceph-exporter.osd-1 on host 'osd-1'
Scheduled to restart crash.osd-0 on host 'osd-0'
Scheduled to restart crash.osd-2 on host 'osd-2'
Scheduled to restart crash.osd-3 on host 'osd-3'
Scheduled to restart crash.osd-1 on host 'osd-1'
Scheduled to restart grafana.osd-0 on host 'osd-0'
Scheduled to restart mds.fsvol001.osd-0.lpciqk on host 'osd-0'
Scheduled to restart mds.fsvol001.osd-2.wocnxz on host 'osd-2'
Scheduled to restart mgr.osd-0.dtkyni on host 'osd-0'
```

```
Scheduled to restart mgr.osd-2.kqcxwu on host 'osd-2'
Scheduled to restart mon.osd-2 on host 'osd-2'
Scheduled to restart mon.osd-3 on host 'osd-3'
Scheduled to restart mon.osd-1 on host 'osd-1'
Scheduled to restart node-exporter.osd-0 on host 'osd-0'
Scheduled to restart node-exporter.osd-2 on host 'osd-2'
Scheduled to restart node-exporter.osd-3 on host 'osd-3'
Scheduled to restart node-exporter.osd-1 on host 'osd-1'
Scheduled to restart osd.1 on host 'osd-0'
Scheduled to restart osd.4 on host 'osd-0'
Scheduled to restart osd.0 on host 'osd-2'
Scheduled to restart osd.5 on host 'osd-2'
Scheduled to restart osd.2 on host 'osd-3'
Scheduled to restart osd.6 on host 'osd-3'
Scheduled to restart osd.3 on host 'osd-1'
Scheduled to restart osd.7 on host 'osd-1'
Scheduled to restart prometheus.osd-0 on host 'osd-0'
Scheduled to restart rgw.rgw.ssl.osd-1.smzpfj on host 'osd-1'
```

Wait for the restarting of all the daemons.

6. Remount existing volumes.
   Depending on your best practices for application maintenance, you can choose the best approach for your environment to remount or remap volumes. One way to remount is to delete the existing application pod and bring up another application pod to use the volume. Another option is to drain the nodes where the applications are running. This ensures that the volume is unmounted from the current pod and then mounted to a new pod, allowing for remapping or remounting of the volume.

4.4. DISABLING ENCRYPTION IN-TRANSIT AFTER DEPLOYMENT IN EXTERNAL MODE

Disable encryption in-transit for OpenShift Data Foundation deployed in external mode by modifying the storage cluster custom resource.

Prerequisites

- OpenShift Data Foundation is deployed and a storage cluster is created.
- Encryption in-transit is enabled for the external mode cluster.

Procedure

1. Remove and check encryption in-transit configurations from the Red Hat Ceph Storage cluster.

```
[root@ceph-client ~]# ceph config rm global ms_client_mode
[root@ceph-client ~]# ceph config rm global ms_cluster_mode
[root@ceph-client ~]# ceph config rm global ms_service_mode
[root@ceph-client ~]# ceph config rm global rbd_default_map_options

[root@ceph-client ~]# ceph config dump | grep ms_
[root@ceph-client ~]#
```

2. Restart all Ceph daemons.
   -

```
[root@ceph-client ~]# ceph orch ls --format plain | tail -n +2 | awk '{print $1}' | xargs -I {} ceph
orch restart {}
Scheduled to restart alertmanager.osd-0 on host 'osd-0'
Scheduled to restart ceph-exporter.osd-0 on host 'osd-0'
Scheduled to restart ceph-exporter.osd-2 on host 'osd-2'
Scheduled to restart ceph-exporter.osd-3 on host 'osd-3'
Scheduled to restart ceph-exporter.osd-1 on host 'osd-1'
Scheduled to restart crash.osd-0 on host 'osd-0'
Scheduled to restart crash.osd-2 on host 'osd-2'
Scheduled to restart crash.osd-3 on host 'osd-3'
Scheduled to restart crash.osd-1 on host 'osd-1'
Scheduled to restart grafana.osd-0 on host 'osd-0'
Scheduled to restart mds.fsvol001.osd-0.lpciqk on host 'osd-0'
Scheduled to restart mds.fsvol001.osd-2.wocnxz on host 'osd-2'
Scheduled to restart mgr.osd-0.dtkyni on host 'osd-0'
Scheduled to restart mgr.osd-2.kqcxwu on host 'osd-2'
Scheduled to restart mon.osd-2 on host 'osd-2'
Scheduled to restart mon.osd-3 on host 'osd-3'
Scheduled to restart mon.osd-1 on host 'osd-1'
Scheduled to restart node-exporter.osd-0 on host 'osd-0'
Scheduled to restart node-exporter.osd-2 on host 'osd-2'
Scheduled to restart node-exporter.osd-3 on host 'osd-3'
Scheduled to restart node-exporter.osd-1 on host 'osd-1'
Scheduled to restart osd.1 on host 'osd-0'
Scheduled to restart osd.4 on host 'osd-0'
Scheduled to restart osd.0 on host 'osd-2'
Scheduled to restart osd.5 on host 'osd-2'
Scheduled to restart osd.6 on host 'osd-3'
Scheduled to restart osd.3 on host 'osd-1'
Scheduled to restart osd.7 on host 'osd-1'
Scheduled to restart prometheus.osd-0 on host 'osd-0'
Scheduled to restart rgw.rgw.ssl.osd-1.bsmbgd on host 'osd-1'
```

```
[root@ceph-client ~]# ceph orch ps
NAME                    HOST   PORTS          STATUS         REFRESHED  AGE  MEM USE
MEM LIM  VERSION          IMAGE ID      CONTAINER ID
alertmanager.osd-0      osd-0  *:9093,9094     running (116s)    9s ago  10h    19.5M       -
0.26.0           7dbf12091920  4694a72d4bbd
ceph-exporter.osd-0     osd-0                  running (19s)     9s ago  10h    7310k       -
18.2.1-229.el9cp  3fd804e38f5b  49bdc7d99471
ceph-exporter.osd-1     osd-1                  running (97s)     26s ago  10h    7285k       -
18.2.1-229.el9cp  3fd804e38f5b  7000d59d23b4
ceph-exporter.osd-2     osd-2                  running (76s)     26s ago  10h    7306k       -
18.2.1-229.el9cp  3fd804e38f5b  3907515cc352
ceph-exporter.osd-3     osd-3                  running (49s)     26s ago  10h    6971k       -
18.2.1-229.el9cp  3fd804e38f5b  3f3952490780
crash.osd-0             osd-0                  running (17s)     9s ago  10h    6878k       -  18.2.1-
229.el9cp  3fd804e38f5b  38e041fb86e3
crash.osd-1             osd-1                  running (96s)     26s ago  10h    6895k       -  18.2.1-
229.el9cp  3fd804e38f5b  21ce3ef7d896
crash.osd-2             osd-2                  running (74s)     26s ago  10h    6899k       -  18.2.1-
229.el9cp  3fd804e38f5b  210ca9c8d928
crash.osd-3             osd-3                  running (47s)     26s ago  10h    6899k       -  18.2.1-
229.el9cp  3fd804e38f5b  710d42d9d138
grafana.osd-0           osd-0  *:3000          running (114s)    9s ago  10h    72.9M       -
10.4.0-pre        f142b583a1b1  3dc5e2248e95
```

```
mds.fsvol001.osd-0.qjntcu  osd-0                running (99s)     9s ago  10h    17.5M       -
18.2.1-229.el9cp  3fd804e38f5b  50efa881c04b
mds.fsvol001.osd-2.qneujv  osd-2                running (51s)     26s ago  10h    15.3M       -
18.2.1-229.el9cp  3fd804e38f5b  a306f2d2d676
mgr.osd-0.zukgyq           osd-0  *:9283,8765,8443  running (21s)     9s ago  10h     442M
-  18.2.1-229.el9cp  3fd804e38f5b  8ef9b728675e
mgr.osd-1.jqfyal           osd-1  *:8443,9283,8765  running (92s)     26s ago  10h     480M       -
18.2.1-229.el9cp  3fd804e38f5b  1ab52db89bfd
mon.osd-1                  osd-1                running (90s)     26s ago  10h    41.7M    2048M
18.2.1-229.el9cp  3fd804e38f5b  88d1fe1e10ac
mon.osd-2                  osd-2                running (72s)     26s ago  10h    31.1M    2048M
18.2.1-229.el9cp  3fd804e38f5b  02f57d3bb44f
mon.osd-3                  osd-3                running (45s)     26s ago  10h    24.0M    2048M
18.2.1-229.el9cp  3fd804e38f5b  5e3783f2b4fa
node-exporter.osd-0        osd-0  *:9100            running (15s)     9s ago  10h    7843k       -
1.7.0             8c904aa522d0  2dae2127349b
node-exporter.osd-1        osd-1  *:9100            running (94s)     26s ago  10h    11.2M       -
1.7.0             8c904aa522d0  010c3fcd55cd
node-exporter.osd-2        osd-2  *:9100            running (69s)     26s ago  10h    17.2M       -
1.7.0             8c904aa522d0  436f2d513f31
node-exporter.osd-3        osd-3  *:9100            running (41s)     26s ago  10h    12.4M       -
1.7.0             8c904aa522d0  5579f0d494b8
osd.0                      osd-0                running (109s)    9s ago  10h     126M    4096M  18.2.1-
229.el9cp  3fd804e38f5b  997076cd39d4
osd.1                      osd-1                running (85s)     26s ago  10h     139M    4096M  18.2.1-
229.el9cp  3fd804e38f5b  08b720f0587d
osd.2                      osd-2                running (65s)     26s ago  10h     143M    4096M  18.2.1-
229.el9cp  3fd804e38f5b  104ad4227163
osd.3                      osd-3                running (36s)     26s ago  10h    94.5M    1435M  18.2.1-
229.el9cp  3fd804e38f5b  db8b265d9f43
osd.4                      osd-0                running (104s)    9s ago  10h     164M    4096M  18.2.1-
229.el9cp  3fd804e38f5b  50dcbbf7e012
osd.5                      osd-1                running (80s)     26s ago  10h     131M    4096M  18.2.1-
229.el9cp  3fd804e38f5b  63b21fe970b5
osd.6                      osd-3                running (32s)     26s ago  10h     243M    1435M  18.2.1-
229.el9cp  3fd804e38f5b  26c7ba208489
osd.7                      osd-2                running (61s)     26s ago  10h     130M    4096M  18.2.1-
229.el9cp  3fd804e38f5b  871a2b75e64f
prometheus.osd-0           osd-0  *:9095            running (12s)     9s ago  10h    44.6M       -
2.48.0            58069186198d  e49a064d2478
rgw.rgw.ssl.osd-1.bsmbgd   osd-1  *:80              running (78s)     26s ago  10h    75.4M       -
18.2.1-229.el9cp  3fd804e38f5b  d03c9f7ae4a4
```

3. Patch the storagecluster to update encryption **enabled** as **false** in the storage cluster spec.

```
$ oc patch storagecluster ocs-external-storagecluster -n openshift-storage --type json --patch
'[{ "op": "replace", "path": "/spec/network", "value": {"connections": {"encryption": {"enabled":
false}}} }]'
storagecluster.ocs.openshift.io/ocs-external-storagecluster patched
```

4. Check the configurations.

```
$ oc get storagecluster
NAME                          AGE   PHASE   EXTERNAL   CREATED AT             VERSION
ocs-external-storagecluster   12h   Ready   true       2024-11-06T20:48:03Z   4.22.0
```

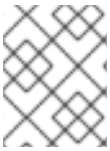
```
$ oc get storagecluster ocs-external-storagecluster -o yaml | yq '.spec.network.connections'
encryption:
  enabled: false
```

5. Remount existing volumes.
   Depending on your best practices for application maintenance, you can choose the best approach for your environment to remount or remap volumes. One way to remount is to delete the existing application pod and bring up another application pod to use the volume. Another option is to drain the nodes where the applications are running. This ensures that the volume is unmounted from the current pod and then mounted to a new pod, allowing for remapping or remounting of the volume.

CHAPTER 5. BLOCK POOLS

Block pools map to storage classes that control how data is stored and replicated in your OpenShift Data Foundation cluster. You can create custom block pools to enable features such as two-replica volumes or compression for specific workloads.

The OpenShift Data Foundation operator installs a default set of storage pools depending on the platform in use. These default storage pools are owned and controlled by the operator and it cannot be deleted or modified.



NOTE

Multiple block pools are not supported for *external mode* OpenShift Data Foundation clusters.

5.1. MANAGING BLOCK POOLS IN INTERNAL MODE

With OpenShift Container Platform, you can create multiple custom storage pools which map to storage classes that provide the following features:

- Enable applications with their own high availability to use persistent volumes with two replicas, potentially improving application performance.
- Save space for persistent volume claims using storage classes with compression enabled.

5.1.1. Creating a block pool

Prerequisites

- You must be logged into the OpenShift Container Platform web console as an administrator.

Procedure

1. Click **Storage → Data Foundation**
2. In the **Storage systems** tab, select the storage system and then click the **Storage pools** tab.
3. Click **Create storage pool**.
4. Select **Volume type** as **Block**.
5. Enter **Pool name**.



NOTE

Using 2-way replication data protection policy is not supported for the default pool. However, you can use 2-way replication if you are creating an additional pool.

6. Select **Data protection policy** as either **Replication** or **Erasure coding**.
   a. Select **Erasure coding** to enable the use of erasure coding instead of replication to protect the cluster data. Supported only for Block (RBD) and File (CephFS).

b. Select the **Scheme** (k+m) for your workload.



NOTE

The minimum number of storage nodes required is equal to the number of data chunks (k) plus the number of parity chunks (m). The recommended minimum is k + m + 1 nodes.

7. Optional: Select **Data compression** checkbox if you need to compress the data.
   Enabling compression can impact application performance and might prove ineffective when data to be written is already compressed or encrypted. Data written before enabling compression is not compressed.

8. Click **Create**.

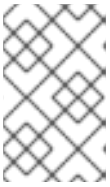
5.1.2. Updating an existing pool

Prerequisites

- You must be logged into the OpenShift Container Platform web console as an administrator.

Procedure

1. Click **Storage → Data Foundation**
2. In the **Storage systems** tab, select the storage system and then click **Storage pools**.
3. Click the Action Menu (⋮) at the end the pool you want to update.
4. Click **Edit storage pool**.
5. Modify the form details as follows:



NOTE

Using 2-way replication data protection policy is not supported for the default pool. However, you can use 2-way replication if you are creating an additional pool.

   a. Change the **Data protection policy** to either 2-way Replication or 3-way Replication.

   b. Enable or disable the compression option.
   Enabling compression can impact application performance and might prove ineffective when data to be written is already compressed or encrypted. Data written before enabling compression is not compressed.

6. Click **Save**.

5.1.3. Deleting a pool

Use this procedure to delete a pool in OpenShift Data Foundation.

Prerequisites

- You must be logged into the OpenShift Container Platform web console as an administrator.

Procedure

1. . Click **Storage → Data Foundation**
2. In the **Storage systems** tab, select the storage system and then click the **Storage pools** tab.
3. Click the Action Menu (⋮) at the end the pool you want to delete.
4. Click **Delete Storage Pool**
5. Click **Delete** to confirm the removal of the Pool.



NOTE

A pool cannot be deleted when it is bound to a PVC. You must detach all the resources before performing this activity.



NOTE

When a pool is deleted, the underlying Ceph pool is not deleted.

CHAPTER 6. CONFIGURE STORAGE FOR OPENSHIFT CONTAINER PLATFORM SERVICES

Configure OpenShift Data Foundation as the storage backend for core OpenShift Container Platform services, including the image registry, monitoring stack, and logging, to ensure reliable and persistent storage for critical cluster operations.

You can use OpenShift Data Foundation to provide storage for OpenShift Container Platform services such as the following:

- OpenShift image registry
- OpenShift monitoring
- OpenShift logging (Loki)

The process for configuring storage for these services depends on the infrastructure used in your OpenShift Data Foundation deployment.



WARNING

Always ensure that you have a plenty of storage capacity for the following OpenShift services that you configure:

- OpenShift image registry
- OpenShift monitoring
- OpenShift logging (Loki)
- OpenShift tracing platform (Tempo)

If the storage for these critical services runs out of space, the OpenShift cluster becomes inoperable and very difficult to recover.

Red Hat recommends configuring shorter curation and retention intervals for these services. See *Configuring the Curator schedule* and *Modifying retention time for Prometheus metrics data* in the Additional resources section for details.

If you do run out of storage space for these services, contact Red Hat Customer Support.

Additional resources

- Configuring the Curator schedule
- Modifying retention time for Prometheus metrics data

6.1. CONFIGURING IMAGE REGISTRY TO USE OPENSHIFT DATA FOUNDATION

OpenShift Container Platform provides a built in Container Image Registry which runs as a standard workload on the cluster. A registry is typically used as a publication target for images built on the cluster as well as a source of images for workloads running on the cluster.

Follow the instructions in this section to configure OpenShift Data Foundation as storage for the Container Image Registry. On AWS, it is not required to change the storage for the registry. However, it is recommended to change the storage to OpenShift Data Foundation Persistent Volume for vSphere and Bare metal platforms.



WARNING

This process does not migrate data from an existing image registry to the new image registry. If you already have container images in your existing registry, back up your registry before you complete this process, and re-register your images when this process is complete.

Prerequisites

- Administrative access to OpenShift Web Console.
- OpenShift Data Foundation Operator is installed and running in the **openshift-storage** namespace. In OpenShift Web Console, click **Ecosystem → Installed Operators** to view installed operators.
- Image Registry Operator is installed and running in the **openshift-image-registry** namespace. In OpenShift Web Console, click **Administration → Cluster Settings → Cluster Operators** to view cluster operators.
- A storage class with provisioner **openshift-storage.cephfs.csi.ceph.com** is available. In OpenShift Web Console, click **Storage → StorageClasses** to view available storage classes.

Procedure

1. **Create a Persistent Volume Claim for the Image Registry to use.**
    a. In the OpenShift Web Console, click **Storage → Persistent Volume Claims**
    b. Set the **Project** to **openshift-image-registry**.
    c. Click **Create Persistent Volume Claim**
        i. From the list of available storage classes retrieved above, specify the **Storage Class** with the provisioner **openshift-storage.cephfs.csi.ceph.com**.
        ii. Specify the Persistent Volume Claim **Name**, for example, **ocs4registry**.
        iii. Specify an **Access Mode** of **Shared Access (RWX)**.
        iv. Specify a **Size** of at least 100 GB.
        v. Click **Create**.

Wait until the status of the new Persistent Volume Claim is listed as **Bound**.

2. **Configure the cluster's Image Registry to use the new Persistent Volume Claim.**
   a. Click **Administration → Custom Resource Definitions**
   b. Click the **Config** custom resource definition associated with the **imageregistry.operator.openshift.io** group.
   c. Click the **Instances** tab.
   d. Beside the cluster instance, click the **Action Menu (⋮) → Edit Config**.
   e. Add the new Persistent Volume Claim as persistent storage for the Image Registry.
      i. Add the following under **spec:**, replacing the existing **storage:** section if necessary.

```
storage:
  pvc:
    claim: <new-pvc-name>
```

      For example:

```
storage:
  pvc:
    claim: ocs4registry
```

      ii. Click **Save**.
3. **Verify that the new configuration is being used.**
   a. Click **Workloads → Pods**.
   b. Set the **Project** to **openshift-image-registry**.
   c. Verify that the new **image-registry-*** pod appears with a status of **Running**, and that the previous **image-registry-*** pod terminates.
   d. Click the new **image-registry-*** pod to view pod details.
   e. Scroll down to **Volumes** and verify that the **registry-storage** volume has a **Type** that matches your new Persistent Volume Claim, for example, **ocs4registry**.

6.2. USING MULTICLOUD OBJECT GATEWAY AS OPENSHIFT IMAGE REGISTRY BACKEND STORAGE

You can use Multicloud Object Gateway (MCG) as OpenShift Container Platform (OCP) Image Registry backend storage in an on-prem OpenShift deployment.

To configure MCG as a backend storage for the OCP image registry, follow the steps mentioned in the procedure.

Prerequisites

- Administrative access to OCP Web Console.

- A running OpenShift Data Foundation cluster with MCG.

Procedure

1. Create **ObjectBucketClaim** by following the steps in Creating Object Bucket Claim .
2. Create an **image-registry-private-configuration-user** secret.
   a. Go to the OpenShift web-console.
   b. Click **ObjectBucketClaim** -→ **ObjectBucketClaim Data**.
   c. In the **ObjectBucketClaim data**, look for **MCG access key** and **MCG secret key** in the **openshift-image-registry namespace**.
   d. Create the secret using the following command:

```
$ oc create secret generic image-registry-private-configuration-user --from-literal=REGISTRY_STORAGE_S3_ACCESSKEY=<MCG Accesskey> --from-literal=REGISTRY_STORAGE_S3_SECRETKEY=<MCG Secretkey> --namespace openshift-image-registry
```

3. Change the status of **managementState** of Image Registry Operator to **Managed**.

```
$ oc patch configs.imageregistry.operator.openshift.io/cluster --type merge -p '{"spec": {"managementState": "Managed"}}'
```

4. Edit the **spec.storage** section of Image Registry Operator configuration file:
   a. Get the **unique-bucket-name** and **regionEndpoint** under the **Object Bucket Claim Data** section from the Web Console **OR** you can also get the information on regionEndpoint and unique-bucket-name from the command:

```
$ oc describe noobaa
```

   b. Add **regionEndpoint** as **http://<Endpoint-name>:<port>** if the
      - storageclass is **ceph-rgw** storageclass and the
      - endpoint points to the internal SVC from the openshift-storage namespace.
   c. An **image-registry** pod spawns after you make the changes to the Operator registry configuration file.

```
$ oc edit configs.imageregistry.operator.openshift.io -n openshift-image-registry
apiVersion: imageregistry.operator.openshift.io/v1
kind: Config
metadata:
[..]
name: cluster
spec:
[..]
storage:
s3:
```

```
bucket: <Unique-bucket-name>

region: us-east-1 (Use this region as default)

regionEndpoint: https://<Endpoint-name>:<port>

virtualHostedStyle: false
```

5. Reset the image registry settings to default.

```
$ oc get pods -n openshift-image-registry
```

Verification

- Run the following command to check if you have configured the MCG as OpenShift Image Registry backend storage successfully.

```
$ oc get pods -n openshift-image-registry
```

Example output

```
$ oc get pods -n openshift-image-registry

NAME                                READY   STATUS      RESTARTS   AGE

cluster-image-registry-operator-56d78bc5fb-bxcgv   2/2     Running     0          44d
image-pruner-1605830400-29r7k                      0/1     Completed   0          10h
image-registry-b6c8f4596-ln88h                     1/1     Running     0          17d
node-ca-2nxvz                                      1/1     Running     0          44d
node-ca-dtwjd                                      1/1     Running     0          44d
node-ca-h92rj                                      1/1     Running     0          44d
node-ca-k9bkd                                      1/1     Running     0          44d
node-ca-stkzc                                      1/1     Running     0          44d
node-ca-xn8h4                                      1/1     Running     0          44d
```

- (Optional) You can also the run the following command to verify if you have configured the MCG as OpenShift Image Registry backend storage successfully.

```
$ oc describe pod <image-registry-name>
```

Example output

```
$ oc describe pod image-registry-b6c8f4596-ln88h

Environment:

    REGISTRY_STORAGE_S3_REGIONENDPOINT:      http://s3.openshift-storage.svc

    REGISTRY_STORAGE:                        s3

    REGISTRY_STORAGE_S3_BUCKET:              bucket-registry-mcg

    REGISTRY_STORAGE_S3_REGION:              us-east-1
```

```
    REGISTRY_STORAGE_S3_ENCRYPT:            true

    REGISTRY_STORAGE_S3_VIRTUALHOSTEDSTYLE:  false

    REGISTRY_STORAGE_S3_USEDUALSTACK:        true

    REGISTRY_STORAGE_S3_ACCESSKEY:           <set to the key
'REGISTRY_STORAGE_S3_ACCESSKEY' in secret 'image-registry-private-configuration'>
Optional: false

    REGISTRY_STORAGE_S3_SECRETKEY:           <set to the key
'REGISTRY_STORAGE_S3_SECRETKEY' in secret 'image-registry-private-configuration'>
Optional: false

    REGISTRY_HTTP_ADDR:                      :5000

    REGISTRY_HTTP_NET:                       tcp

    REGISTRY_HTTP_SECRET:
57b943f691c878e342bac34e657b702bd6ca5488d51f839fecafa918a79a5fc6ed70184cab04760
1403c1f383e54d458744062dcaaa483816d82408bb56e686f

    REGISTRY_LOG_LEVEL:                      info

    REGISTRY_OPENSHIFT_QUOTA_ENABLED:        true

    REGISTRY_STORAGE_CACHE_BLOBDESCRIPTOR:   inmemory

    REGISTRY_STORAGE_DELETE_ENABLED:         true

    REGISTRY_OPENSHIFT_METRICS_ENABLED:      true

    REGISTRY_OPENSHIFT_SERVER_ADDR:          image-registry.openshift-image-
registry.svc:5000

    REGISTRY_HTTP_TLS_CERTIFICATE:           /etc/secrets/tls.crt

    REGISTRY_HTTP_TLS_KEY:                   /etc/secrets/tls.key
```

6.3. CONFIGURING MONITORING TO USE OPENSHIFT DATA FOUNDATION

OpenShift Data Foundation provides a monitoring stack that comprises of Prometheus and Alert Manager.

Follow the instructions in this section to configure OpenShift Data Foundation as storage for the monitoring stack.



IMPORTANT

Monitoring will not function if it runs out of storage space. Always ensure that you have plenty of storage capacity for monitoring.

Red Hat recommends configuring a short retention interval for this service. See the Modifying retention time for Prometheus metrics data of Monitoring guide in the OpenShift Container Platform documentation for details.

Prerequisites

- Administrative access to OpenShift Web Console.
- OpenShift Data Foundation Operator is installed and running in the **openshift-storage** namespace. In the OpenShift Web Console, click **Ecosystem → Installed Operators** to view installed operators.
- Monitoring Operator is installed and running in the **openshift-monitoring** namespace. In the OpenShift Web Console, click **Administration → Cluster Settings → Cluster Operators** to view cluster operators.
- A storage class with provisioner **openshift-storage.rbd.csi.ceph.com** is available. In the OpenShift Web Console, click **Storage → StorageClasses** to view available storage classes.

Procedure

1. In the OpenShift Web Console, go to **Workloads → Config Maps**.
2. Set the **Project** dropdown to **openshift-monitoring**.
3. Click **Create Config Map**.
4. Define a new **cluster-monitoring-config** Config Map using the following example. Replace the content in angle brackets (**<**, **>**) with your own values, for example, **retention: 24h** or **storage: 40Gi**.

   Replace the **storageClassName** with the **storageclass** that uses the provisioner **openshift-storage.rbd.csi.ceph.com**. In the example given below the name of the **storageclass** is **ocs-storagecluster-ceph-rbd**.

   Example cluster-monitoring-config Config Map

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-monitoring-config
  namespace: openshift-monitoring
data:
  config.yaml: |
    prometheusK8s:
      retention: <time to retain monitoring files, for example 24h>
      volumeClaimTemplate:
        metadata:
          name: ocs-prometheus-claim
        spec:
```

```
      storageClassName: ocs-storagecluster-ceph-rbd
      resources:
        requests:
          storage: <size of claim, e.g. 40Gi>
  alertmanagerMain:
    volumeClaimTemplate:
      metadata:
        name: ocs-alertmanager-claim
      spec:
        storageClassName: ocs-storagecluster-ceph-rbd
        resources:
          requests:
            storage: <size of claim, e.g. 40Gi>
```

1. Click **Create** to save and create the Config Map.

Verification

1. Verify that the Persistent Volume Claims are bound to the pods.
   a. Go to **Storage → Persistent Volume Claims**
   b. Set the **Project** dropdown to **openshift-monitoring**.
   c. Verify that 5 Persistent Volume Claims are visible with a state of **Bound**, attached to three **alertmanager-main-*** pods, and two **prometheus-k8s-*** pods.
   Monitoring storage created and bound image::images/openshift-monitoring-pvcs-created.png["Screenshot of OpenShift Web Console showing five pods with persistent volume claims bound in the openshift-monitoring project"]
2. Verify that the new **alertmanager-main-*** pods appear with a state of **Running**.
   a. Go to **Workloads → Pods**.
   b. Click the new **alertmanager-main-*** pods to view the pod details.
   c. Scroll down to **Volumes** and verify that the volume has a **Type**, **ocs-alertmanager-claim** that matches one of your new Persistent Volume Claims, for example, **ocs-alertmanager-claim-alertmanager-main-0**.
   Persistent Volume Claims attached to alertmanager-main-* pod* image::images/openshift-monitoring-alert-pods-attached.png["Screenshot of OpenShift Web Console showing persistent volume claim attached to the altermanager pod"]
3. Verify that the new **prometheus-k8s-*** pods appear with a state of **Running**.
   a. Click the new **prometheus-k8s-*** pods to view the pod details.
   b. Scroll down to **Volumes** and verify that the volume has a **Type**, **ocs-prometheus-claim** that matches one of your new Persistent Volume Claims, for example, **ocs-prometheus-claim-prometheus-k8s-0**.
   Persistent Volume Claims attached to prometheus-k8s-* pod* image::images/openshift-monitoring-prometheus-pods-attached.png["Screenshot of OpenShift Web Console showing persistent volume claim attached to the prometheus pod"]

6.4. OVERPROVISION LEVEL POLICY CONTROL

Overprovision control is a mechanism that enables you to define a quota on the amount of Persistent Volume Claims (PVCs) consumed from a storage cluster, based on the specific application namespace.

When you enable the overprovision control mechanism, it prevents you from overprovisioning the PVCs consumed from the storage cluster. OpenShift provides flexibility for defining constraints that limit the aggregated resource consumption at cluster scope with the help of **ClusterResourceQuota**. For more information see, OpenShift ClusterResourceQuota.

With overprovision control, a **ClusteResourceQuota** is initiated, and you can set the storage capacity limit for each storage class.

For more information about OpenShift Data Foundation deployment, refer to Product Documentation and select the deployment procedure according to the platform.

Configuring the quota limit to receive overprovision control alerts

Prerequisites

- Ensure that the OpenShift Data Foundation cluster is created.

Procedure

1. Deploy **storagecluster** either from the command line interface or the user interface.
2. Label the application namespace.

```
apiVersion: v1
kind: Namespace
metadata:
    name: <desired_name>
    labels:
        storagequota: <desired_label>
```

<desired_name>

Specify a name for the application namespace, for example, **quota-rbd**.

<desired_label>

Specify a label for the storage quota, for example, **storagequota1**.

3. Edit the **storagecluster** to set the quota limit on the storage class.

```
$ oc edit storagecluster -n openshift-storage <ocs_storagecluster_name>
```

<ocs_storagecluster_name>

Specify the name of the storage cluster.

4. Add an entry for Overprovision Control with the desired hard limit into the **StorageCluster.Spec**:

```
apiVersion: ocs.openshift.io/v1
kind: StorageCluster
spec:
 [...]
    overprovisionControl:
    - capacity: <desired_quota_limit>
```

```
        storageClassName: <storage_class_name>
        quotaName: <desired_quota_name>
        selector:
          labels:
            matchLabels:
              storagequota: <desired_label>
[...]
```

<desired_quota_limit>

Specify a desired quota limit for the storage class, for example, **27Ti**.

<storage_class_name>

Specify the name of the storage class for which you want to set the quota limit, for example, **ocs-storagecluster-ceph-rbd**.

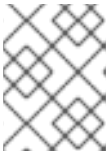
<desired_quota_name>

Specify a name for the storage quota, for example, **quota1**.

<desired_label>

Specify a label for the storage quota, for example, **storagequota1**.

5. Save the modified **storagecluster**.
6. Verify that the **clusterresourcequota** is defined.



NOTE

Expect the **clusterresourcequota** with the **quotaName** that you defined in the previous step, for example, **quota1**.

```
$ oc get clusterresourcequota -A

$ oc describe clusterresourcequota -A
```

6.5. CLUSTER LOGGING FOR OPENSHIFT DATA FOUNDATION

Deploy cluster logging with OpenShift Data Foundation as the storage backend to aggregate and retain logs for OpenShift Container Platform services using the Loki log store.

You can deploy cluster logging to aggregate logs for a range of OpenShift Container Platform services.

For information about how to install and configure the cluster logging operator, see *Installing Red Hat OpenShift Logging Operator* and *Loki object storage* in the Additional resources section.



IMPORTANT

If there is no retention period defined on the s3 bucket or in the LokiStack custom resource (CR), then the logs are not pruned and they stay in the s3 bucket forever, which might fill up the s3 storage. For directions to configure retention policies, see *Enabling stream-based retention with Loki* in the Additional resources section.

Additional resources

- Installing Red Hat OpenShift Logging Operator

- Loki object storage
- Enabling stream-based retention with Loki

CHAPTER 7. BACKING OPENSHIFT CONTAINER PLATFORM APPLICATIONS WITH OPENSHIFT DATA FOUNDATION

You cannot directly install OpenShift Data Foundation during the OpenShift Container Platform installation. However, you can install OpenShift Data Foundation on an existing OpenShift Container Platform by using the Software Catalog and then configure the OpenShift Container Platform applications to be backed by OpenShift Data Foundation.

Prerequisites

- OpenShift Container Platform is installed and you have administrative access to OpenShift Web Console.
- OpenShift Data Foundation is installed and running in the **openshift-storage** namespace.

Procedure

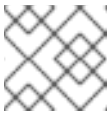
1. In the OpenShift Web Console, perform one of the following:
   - Click **Workloads → Deployments**.
     In the Deployments page, you can do one of the following:
     - Select any existing deployment and click **Add Storage** option from the **Action** menu (⋮).
     - Create a new deployment and then add storage.
       i. Click **Create Deployment** to create a new deployment.
       ii. Edit the **YAML** based on your requirement to create a deployment.
       iii. Click **Create**.
       iv. Select **Add Storage** from the **Actions** drop-down menu on the top right of the page.
   - Click **Workloads → Deployment Configs**
     In the Deployment Configs page, you can do one of the following:
     - Select any existing deployment and click **Add Storage** option from the **Action** menu (⋮).
     - Create a new deployment and then add storage.
       i. Click **Create Deployment Config** to create a new deployment.
       ii. Edit the **YAML** based on your requirement to create a deployment.
       iii. Click **Create**.
       iv. Select **Add Storage** from the **Actions** drop-down menu on the top right of the page.
2. In the Add Storage page, you can choose one of the following options:
   - Click the **Use existing claim** option and select a suitable PVC from the drop-down list.

- Click the **Create new claim** option.

  a. Select the appropriate **CephFS** or **RBD** storage class from the **Storage Class** drop-down list.

  b. Provide a name for the Persistent Volume Claim.

  c. Select ReadWriteOnce (RWO) or ReadWriteMany (RWX) access mode.

  NOTE

  ReadOnlyMany (ROX) is deactivated as it is not supported.

  d. Select the size of the desired storage capacity.

  NOTE

  You can expand the block PVs but cannot reduce the storage capacity after the creation of Persistent Volume Claim.

3. Specify the mount path and subpath (if required) for the mount path volume inside the container.

4. Click **Save**.

Verification

1. Depending on your configuration, perform one of the following:

   - Click **Workloads → Deployments**.
   - Click **Workloads → Deployment Configs**

2. Set the Project as required.

3. Click the deployment for which you added storage to display the deployment details.

4. Scroll down to **Volumes** and verify that your deployment has a **Type** that matches the Persistent Volume Claim that you assigned.

5. Click the Persistent Volume Claim name and verify the storage class name in the Persistent Volume Claim Overview page.

CHAPTER 8. ADDING FILE AND OBJECT STORAGE TO AN EXISTING EXTERNAL OPENSHIFT DATA FOUNDATION CLUSTER

When OpenShift Data Foundation is configured in external mode, there are several ways to provide storage for persistent volume claims and object bucket claims.

- Persistent volume claims for block storage are provided directly from the external Red Hat Ceph Storage cluster.
- Persistent volume claims for file storage can be provided by adding a Metadata Server (MDS) to the external Red Hat Ceph Storage cluster.
- Object bucket claims for object storage can be provided either by using the Multicloud Object Gateway or by adding the Ceph Object Gateway to the external Red Hat Ceph Storage cluster.

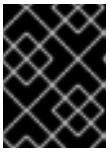
Use the following process to add file storage (using Metadata Servers) or object storage (using Ceph Object Gateway) or both to an external OpenShift Data Foundation cluster that was initially deployed to provide only block storage.

Prerequisites

- OpenShift Data Foundation 4.22 is installed and running on the OpenShift Container Platform version 4.22 or above. Also, the OpenShift Data Foundation Cluster in external mode is in the **Ready** state.
- Your external Red Hat Ceph Storage cluster is configured with one or both of the following:
  - a Ceph Object Gateway (RGW) endpoint that can be accessed by the OpenShift Container Platform cluster for object storage
  - a Metadata Server (MDS) pool for file storage
- Ensure that you know the parameters used with the **ceph-external-cluster-details-exporter.py** script during external OpenShift Data Foundation cluster deployment.

Procedure

1. Download the OpenShift Data Foundation version of the **ceph-external-cluster-details-exporter.py** python script using one of the following methods, either CSV or ConfigMap.



IMPORTANT

Downloading the **ceph-external-cluster-details-exporter.py** python script using CSV is no longer supported. Using the ConfigMap is the only supported method.

CSV

```
# oc get csv $(oc get csv -n openshift-storage | grep rook-ceph-operator | awk '{print $1}') -n openshift-storage -o jsonpath='{.metadata.annotations.externalClusterScript}' | base64 --decode > ceph-external-cluster-details-exporter.py
```

ConfigMap

```
# oc get cm rook-ceph-external-cluster-script-config -n openshift-storage -o
jsonpath='{.data.script}' | base64 --decode > ceph-external-cluster-details-exporter.py
```

2. Update permission caps on the external Red Hat Ceph Storage cluster by running **ceph-external-cluster-details-exporter.py** on any client node in the external Red Hat Ceph Storage cluster. You may need to ask your Red Hat Ceph Storage administrator to do this.

```
# python3 ceph-external-cluster-details-exporter.py --upgrade \
--run-as-user=__ocs-client-name__ \
--rgw-pool-prefix __rgw-pool-prefix__
```

--run-as-user

The client name used during OpenShift Data Foundation cluster deployment. Use the default client name **client.healthchecker** if a different client name was not set.

--rgw-pool-prefix

The prefix used for the Ceph Object Gateway pool. This can be omitted if the default prefix is used.

3. Generate and save configuration details from the external Red Hat Ceph Storage cluster.

   a. Generate configuration details by running **ceph-external-cluster-details-exporter.py** on any client node in the external Red Hat Ceph Storage cluster.

```
# python3 ceph-external-cluster-details-exporter.py --rbd-data-pool-name __rbd-block-
pool-name__ --monitoring-endpoint __ceph-mgr-prometheus-exporter-endpoint__ --
monitoring-endpoint-port __ceph-mgr-prometheus-exporter-port__ --run-as-user __ocs-
client-name__  --rgw-endpoint __rgw-endpoint__ --rgw-pool-prefix __rgw-pool-prefix__
```

--monitoring-endpoint

Is optional. It accepts comma separated list of IP addresses of active and standby mgrs reachable from the OpenShift Container Platform cluster. If not provided, the value is automatically populated.

--monitoring-endpoint-port

Is optional. It is the port associated with the ceph-mgr Prometheus exporter specified by **--monitoring-endpoint**. If not provided, the value is automatically populated.

--run-as-user

The client name used during OpenShift Data Foundation cluster deployment. Use the default client name **client.healthchecker** if a different client name was not set.

--rgw-endpoint

Provide this parameter to provision object storage through Ceph Object Gateway for OpenShift Data Foundation. (optional parameter)

--rgw-pool-prefix

The prefix used for the Ceph Object Gateway pool. This can be omitted if the default prefix is used.

User permissions are updated as shown:

```
caps: [mgr] allow command config
caps: [mon] allow r, allow command quorum_status, allow command version
caps: [osd] allow rwx pool=default.rgw.meta, allow r pool=.rgw.root, allow rw
```

```
pool=default.rgw.control, allow rx pool=default.rgw.log, allow x
pool=default.rgw.buckets.index
```



NOTE

Ensure that all the parameters (including the optional arguments) except the Ceph Object Gateway details (if provided), are the same as what was used during the deployment of OpenShift Data Foundation in external mode.

b. Save the output of the script in an **external-cluster-config.json** file.
The following example output shows the generated configuration changes in bold text.

```
[{"name": "rook-ceph-mon-endpoints", "kind": "ConfigMap", "data": {"data":
"xxx.xxx.xxx.xxx:xxxx", "maxMonId": "0", "mapping": "{}"}}, {"name": "rook-ceph-mon",
"kind": "Secret", "data": {"admin-secret": "admin-secret", "fsid": "<fs-id>", "mon-secret":
"mon-secret"}}, {"name": "rook-ceph-operator-creds", "kind": "Secret", "data": {"userID": "
<user-id>", "userKey": "<user-key>"}}, {"name": "rook-csi-rbd-node", "kind": "Secret",
"data": {"userID": "csi-rbd-node", "userKey": "<user-key>"}}, {"name": "ceph-rbd", "kind":
"StorageClass", "data": {"pool": "<pool>"}}, {"name": "monitoring-endpoint", "kind":
"CephCluster", "data": {"MonitoringEndpoint": "xxx.xxx.xxx.xxx", "MonitoringPort":
"xxxx"}}, {"name": "rook-ceph-dashboard-link", "kind": "Secret", "data": {"userID": "ceph-
dashboard-link", "userKey": "<user-key>"}}, {"name": "rook-csi-rbd-provisioner", "kind":
"Secret", "data": {"userID": "csi-rbd-provisioner", "userKey": "<user-key>"}}, {"name":
"rook-csi-cephfs-provisioner", "kind": "Secret", "data": {"adminID": "csi-cephfs-
provisioner", "adminKey": "<admin-key>"}}, {"name": "rook-csi-cephfs-node", "kind":
"Secret", "data": {"adminID": "csi-cephfs-node", "adminKey": "<admin-key>"}}, {"name":
"cephfs", "kind": "StorageClass", "data": {"fsName": "cephfs", "pool": "cephfs_data"}},
{"name": "ceph-rgw", "kind": "StorageClass", "data": {"endpoint": "xxx.xxx.xxx.xxx:xxxx",
"poolPrefix": "default"}}, {"name": "rgw-admin-ops-user", "kind": "Secret", "data":
{"accessKey": "<access-key>", "secretKey": "<secret-key>"}}]
```

4. Upload the generated JSON file.

   a. Log in to the OpenShift web console.

   b. Click **Workloads → Secrets**.

   c. Set **project** to **openshift-storage**.

   d. Click on **rook-ceph-external-cluster-details**.

   e. Click **Actions** (⋮) → **Edit Secret**

   f. Click **Browse** and upload the **external-cluster-config.json** file.

   g. Click **Save**.

Verification

- To verify that the OpenShift Data Foundation cluster is healthy and data is resilient, navigate to **Storage → Data foundation → Storage Systems** tab and then click on the storage system name.
  - On the **Overview → Block and File** tab, check the Status card to confirm that the *Storage Cluster* has a green tick indicating it is healthy.

- If you added a Metadata Server for file storage:

  a. Click **Workloads → Pods** and verify that **openshift-storage.cephfs.csi.ceph.com-nodeplugin-*** and **openshift-storage.cephfs.csi.ceph.com-ctrlplugin-*** pods are created new and are in the **Running** state.

  b. Click **Storage → Storage Classes** and verify that the **ocs-external-storagecluster-cephfs** storage class is created.

- If you added the Ceph Object Gateway for object storage:

  a. Click **Storage → Storage Classes** and verify that the **ocs-external-storagecluster-ceph-rgw** storage class is created.

  b. To verify that the OpenShift Data Foundation cluster is healthy and data is resilient, navigate to **Storage → Data foundation → Storage Systems** tab and then click on the storage system name.

  c. Click the **Object** tab and confirm *Object Service* and *Data resiliency* has a green tick indicating it is healthy.

CHAPTER 9. HOW TO USE DEDICATED WORKER NODES FOR RED HAT OPENSHIFT DATA FOUNDATION

Any Red Hat OpenShift Container Platform subscription requires an OpenShift Data Foundation subscription. However, you can save on the OpenShift Container Platform subscription costs if you are using infrastructure nodes to schedule OpenShift Data Foundation resources.

It is important to maintain consistency across environments with or without Machine API support. Because of this, it is highly recommended in all cases to have a special category of nodes labeled as either worker or infra or have both roles. See the *Manual creation of infrastructure nodes* section for more information.

9.1. ANATOMY OF AN INFRASTRUCTURE NODE

Infrastructure nodes for use with OpenShift Data Foundation have a few attributes. The **infra** node-role label is required to ensure the node does not consume RHOCP entitlements. The **infra** node-role label is responsible for ensuring only OpenShift Data Foundation entitlements are necessary for the nodes running OpenShift Data Foundation.

- Labeled with **node-role.kubernetes.io/infra**

Adding an OpenShift Data Foundation taint with a **NoSchedule** effect is also required so that the **infra** node will only schedule OpenShift Data Foundation resources.

- Tainted with **node.ocs.openshift.io/storage="true"**

The label identifies the RHOCP node as an **infra** node so that RHOCP subscription cost is not applied. The taint prevents non OpenShift Data Foundation resources to be scheduled on the tainted nodes.



NOTE

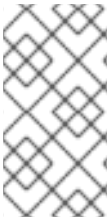
Adding storage taint on nodes might require toleration handling for the other **daemonset** pods such as **openshift-dns daemonset**. For information about how to manage the tolerations, see the *Openshift-dns daemonsets tolerations* knowledge base article.

Example of the taint and labels required on infrastructure node that will be used to run OpenShift Data Foundation services:

```
spec:
  taints:
  - effect: NoSchedule
    key: node.ocs.openshift.io/storage
    value: "true"
  metadata:
    creationTimestamp: null
    labels:
      node-role.kubernetes.io/worker: ""
      node-role.kubernetes.io/infra: ""
      cluster.ocs.openshift.io/openshift-storage: ""
```

9.2. MACHINE SETS FOR CREATING INFRASTRUCTURE NODES

If the Machine API is supported in the environment, then labels should be added to the templates for the Machine Sets that will be provisioning the infrastructure nodes. Avoid the anti-pattern of adding labels manually to nodes created by the machine API. Doing so is analogous to adding labels to pods created by a deployment. In both cases, when the pod/node fails, the replacement pod/node will not have the appropriate labels.

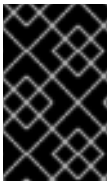


NOTE

In EC2 environments, you will need three machine sets, each configured to provision infrastructure nodes in a distinct availability zone (such as us-east-2a, us-east-2b, us-east-2c). Currently, OpenShift Data Foundation does not support deploying in more than three availability zones.

The following Machine Set template example creates nodes with the appropriate taint and labels required for infrastructure nodes. This will be used to run OpenShift Data Foundation services.

```
template:
  metadata:
    creationTimestamp: null
    labels:
      machine.openshift.io/cluster-api-cluster: kb-s25vf
      machine.openshift.io/cluster-api-machine-role: worker
      machine.openshift.io/cluster-api-machine-type: worker
      machine.openshift.io/cluster-api-machineset: kb-s25vf-infra-us-west-2a
  spec:
    taints:
    - effect: NoSchedule
      key: node.ocs.openshift.io/storage
      value: "true"
    metadata:
      creationTimestamp: null
      labels:
        node-role.kubernetes.io/infra: ""
        cluster.ocs.openshift.io/openshift-storage: ""
```



IMPORTANT

If you add a taint to the infrastructure nodes, you also need to add tolerations to the taint for other workloads, for example, the fluentd pods. For more information, see the *Infrastructure Nodes in OpenShift 4* knowledge base article.

9.3. MANUAL CREATION OF INFRASTRUCTURE NODES

Only when the Machine API is not supported in the environment should labels be directly applied to nodes. Manual creation requires that at least 3 RHOCP worker nodes are available to schedule OpenShift Data Foundation services, and that these nodes have sufficient CPU and memory resources. To avoid the RHOCP subscription cost, the following is required:

```
oc label node <node> node-role.kubernetes.io/infra=""
oc label node <node> cluster.ocs.openshift.io/openshift-storage=""
```

Adding a **NoSchedule** OpenShift Data Foundation taint is also required so that the **infra** node will only schedule OpenShift Data Foundation resources and repel any other non-OpenShift Data Foundation workloads.

```
oc adm taint node <node> node.ocs.openshift.io/storage="true":NoSchedule
```



WARNING

Do not remove the node-role node-role.kubernetes.io/worker=""

The removal of the **node-role.kubernetes.io/worker=""** can cause issues unless changes are made both to the OpenShift scheduler and to MachineConfig resources.

If already removed, it should be added again to each **infra** node. Adding node-role **node-role.kubernetes.io/infra=""** and OpenShift Data Foundation taint is sufficient to conform to entitlement exemption requirements.

9.4. TAINTING A NODE FROM THE USER INTERFACE

Taint OpenShift Data Foundation nodes from the OpenShift web console after deployment to dedicate them exclusively to storage workloads.

This section explains the procedure to taint nodes after the OpenShift Data Foundation deployment.

Procedure

1. In the OpenShift Web Console, click **Compute → Nodes**, and then select the node which has to be tainted.
2. In the **Details** page click on **Edit taints**.
3. Enter the values in the **Key** <nodes.openshift.ocs.io/storage>, **Value** <true> and in the **Effect**<Noschedule> field.
4. Click Save.

Verification

- Follow the steps to verify that the node has tainted successfully:
  - Navigate to **Compute → Nodes**.
  - Select the node to verify its status, and then click on the **YAML** tab.
  - In the **specs** section check the values of the following parameters:

```
Taints:
  Key: node.openshift.ocs.io/storage
  Value: true
```

Effect: Noschedule

Additional resources

- Creating the OpenShift Data Foundation cluster on VMware vSphere

CHAPTER 10. MANAGING PERSISTENT VOLUME CLAIMS

Manage persistent volume claims (PVCs) to provision, monitor, and expand OpenShift Data Foundation storage for your application workloads running on OpenShift Container Platform.

10.1. CONFIGURING APPLICATION PODS TO USE OPENSHIFT DATA FOUNDATION

Follow the instructions in this section to configure OpenShift Data Foundation as storage for an application pod.

Prerequisites

- Administrative access to OpenShift Web Console.
- OpenShift Data Foundation Operator is installed and running in the **openshift-storage** namespace. In OpenShift Web Console, click **Ecosystem → Installed Operators** to view installed operators.
- The default storage classes provided by OpenShift Data Foundation are available. In OpenShift Web Console, click **Storage → StorageClasses** to view default storage classes.

Procedure

1. **Create a Persistent Volume Claim (PVC) for the application to use.**
   a. In OpenShift Web Console, click **Storage → Persistent Volume Claims**
   b. Set the **Project** for the application pod.
   c. Click **Create Persistent Volume Claim**
      i. Specify a **Storage Class** provided by OpenShift Data Foundation.
      ii. Specify the PVC **Name**, for example, **myclaim**.
      iii. Select the required **Access Mode**.



NOTE

The **Access Mode**, **Shared access (RWX)** is not supported in IBM FlashSystem.

      iv. For Rados Block Device (RBD), if the **Access mode** is ReadWriteOnce (**RWO**), select the required **Volume mode**. The default volume mode is **Filesystem**.
      v. Specify a **Size** as per application requirement.
      vi. Click **Create** and wait until the PVC is in **Bound** status.
2. **Configure a new or existing application pod to use the new PVC.**
   - For a new application pod, perform the following steps:
      i. Click **Workloads →Pods**.

ii. Create a new application pod.

iii. Under the **spec:** section, add **volumes:** section to add the new PVC as a volume for the application pod.

```
volumes:
  - name: <volume_name>
    persistentVolumeClaim:
      claimName: <pvc_name>
```

For example:

```
volumes:
  - name: mypd
    persistentVolumeClaim:
      claimName: myclaim
```

- For an existing application pod, perform the following steps:

i. Click **Workloads →Deployment Configs**.

ii. Search for the required deployment config associated with the application pod.

iii. Click on its **Action menu (⋮) → Edit Deployment Config**.

iv. Under the **spec:** section, add **volumes:** section to add the new PVC as a volume for the application pod and click **Save**.

```
volumes:
  - name: <volume_name>
    persistentVolumeClaim:
      claimName: <pvc_name>
```

For example:

```
volumes:
  - name: mypd
    persistentVolumeClaim:
      claimName: myclaim
```

3. **Verify that the new configuration is being used.**

a. Click **Workloads → Pods**.

b. Set the **Project** for the application pod.

c. Verify that the application pod appears with a status of **Running**.

d. Click the application pod name to view pod details.

e. Scroll down to **Volumes** section and verify that the volume has a **Type** that matches your new Persistent Volume Claim, for example, **myclaim**.

10.2. VIEWING PERSISTENT VOLUME CLAIM REQUEST STATUS

Use this procedure to view the status of a PVC request.

Prerequisites

- Administrator access to OpenShift Data Foundation.

Procedure

1. Log in to OpenShift Web Console.
2. Click **Storage → Persistent Volume Claims**
3. Search for the required PVC name by using the **Filter** textbox. You can also filter the list of PVCs by Name or Label to narrow down the list
4. Check the **Status** column corresponding to the required PVC.
5. Click the required **Name** to view the PVC details.

10.3. REVIEWING PERSISTENT VOLUME CLAIM REQUEST EVENTS

Use this procedure to review and address Persistent Volume Claim (PVC) request events.

Prerequisites

- Administrator access to OpenShift Web Console.

Procedure

1. In the OpenShift Web Console, click **Storage → Data Foundation**
2. In the **Storage systems** tab, select the storage system and then click **Overview → Block and File**.
3. Locate the **Inventory** card to see the number of PVCs with errors.
4. Click **Storage → Persistent Volume Claims**
5. Search for the required PVC using the **Filter** textbox.
6. Click on the PVC name and navigate to **Events**
7. Address the events as required or as directed.

10.4. EXPANDING PERSISTENT VOLUME CLAIMS

OpenShift Data Foundation 4.6 onwards has the ability to expand Persistent Volume Claims providing more flexibility in the management of persistent storage resources.

Expansion is supported for the following Persistent Volumes:

- PVC with ReadWriteOnce (RWO) and ReadWriteMany (RWX) access that is based on Ceph File System (CephFS) for volume mode **Filesystem**.

- PVC with ReadWriteOnce (RWO) access that is based on Ceph RADOS Block Devices (RBDs) with volume mode **Filesystem**.
- PVC with ReadWriteOnce (RWO) access that is based on Ceph RADOS Block Devices (RBDs) with volume mode **Block**.
- PVC with ReadWriteOncePod (RWOP) that is based on Ceph File System (CephFS) or Network File System (NFS) for volume mode **Filesystem**.
- PVC with ReadWriteOncePod (RWOP) access that is based on Ceph RADOS Block Devices (RBDs) with volume mode **Filesystem**. With RWOP access mode, you mount the volume as read-write by a single pod on a single node.



NOTE

PVC expansion is not supported for MON.

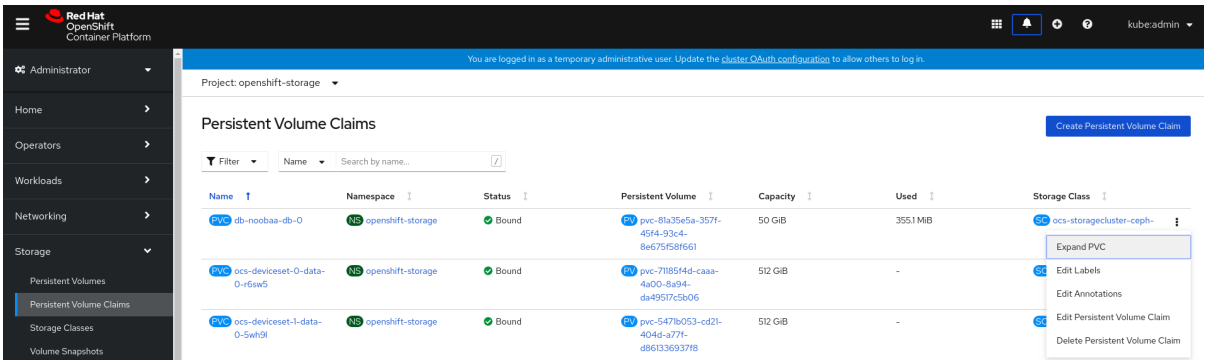
PVC expansion is supported for RBD, OSD, and encrypted PVCs.

Prerequisites

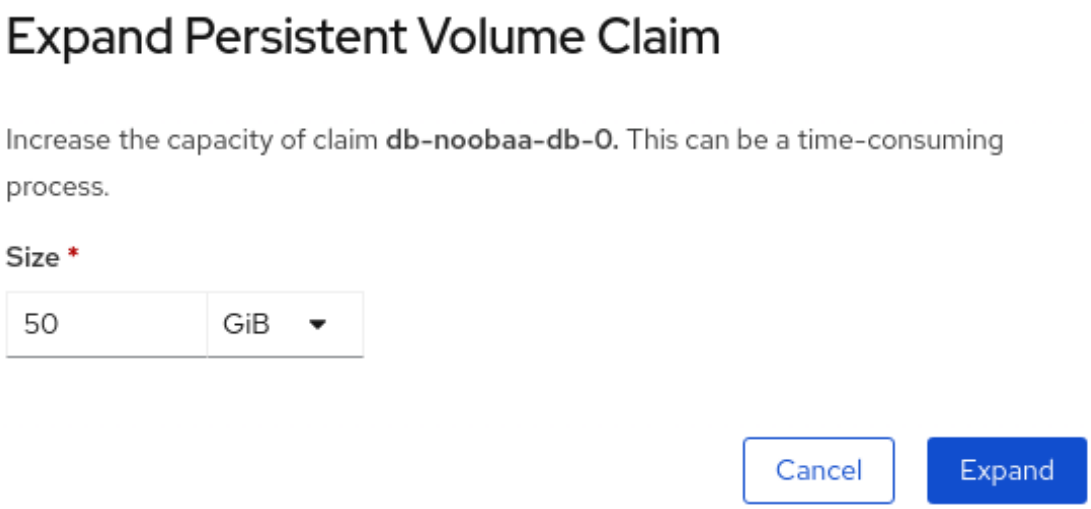
- Administrator access to OpenShift Web Console.

Procedure

1. In OpenShift Web Console, navigate to **Storage → Persistent Volume Claims**.
2. Click the Action Menu (⋮) next to the Persistent Volume Claim you want to expand.
3. Click **Expand PVC**:



4. Select the new size of the Persistent Volume Claim, then click **Expand**:



5. To verify the expansion, navigate to the PVC's details page and verify the **Capacity** field has the correct size requested.



NOTE

When expanding PVCs based on Ceph RADOS Block Devices (RBDs), if the PVC is not already attached to a pod the **Condition type** is **FileSystemResizePending** in the PVC's details page. Once the volume is mounted, filesystem resize succeeds and the new size is reflected in the **Capacity** field.

10.5. DYNAMIC PROVISIONING

Dynamic provisioning allows storage volumes to be automatically created on demand when an application requests persistent storage, removing the need for administrators to pre-provision storage manually.

10.5.1. About dynamic provisioning

The StorageClass resource object describes and classifies storage that can be requested, as well as provides a means for passing parameters for dynamically provisioned storage on demand. StorageClass objects can also serve as a management mechanism for controlling different levels of storage and access to the storage. Cluster Administrators (**cluster-admin**) or Storage Administrators (**storage-admin**) define and create the StorageClass objects that users can request without needing any intimate knowledge about the underlying storage volume sources.

The OpenShift Data Foundation persistent volume framework enables this functionality and allows administrators to provision a cluster with persistent storage. The framework also gives users a way to request those resources without having any knowledge of the underlying infrastructure.

Many storage types are available for use as persistent volumes in OpenShift Data Foundation. Storage plug-ins might support static provisioning, dynamic provisioning or both provisioning types.

10.5.2. Dynamic provisioning in OpenShift Data Foundation

Red Hat OpenShift Data Foundation is software-defined storage that is optimised for container environments. It runs as an operator on OpenShift Container Platform to provide highly integrated and simplified persistent storage management for containers.

OpenShift Data Foundation supports a variety of storage types, including:

- Block storage for databases
- Shared file storage for continuous integration, messaging, and data aggregation
- Object storage for archival, backup, and media storage

Version 4 uses Red Hat Ceph Storage to provide the file, block, and object storage that backs persistent volumes, and Rook.io to manage and orchestrate provisioning of persistent volumes and claims. NooBaa provides object storage, and its Multicloud Gateway allows object federation across multiple cloud environments (available as a Technology Preview).

In OpenShift Data Foundation 4, the Red Hat Ceph Storage Container Storage Interface (CSI) driver for RADOS Block Device (RBD) and Ceph File System (CephFS) handles the dynamic provisioning requests. When a PVC request comes in dynamically, the CSI driver has the following options:

- Create a PVC with ReadWriteOnce (RWO) and ReadWriteMany (RWX) access that is based on Ceph RBDs with volume mode **Block**.
- Create a PVC with ReadWriteOnce (RWO) access that is based on Ceph RBDs with volume mode **Filesystem**. The supported **fstype** for **Filesystem** mode is **ext4**. This is the default.
- Create a PVC with ReadWriteOnce (RWO) and ReadWriteMany (RWX) access that is based on CephFS for volume mode **Filesystem**.
- Create a PVC with ReadWriteOncePod (RWOP) access that is based on CephFS,NFS and RBD. With RWOP access mode, you mount the volume as read-write by a single pod on a single node.

The judgment of which driver (RBD or CephFS) to use is based on the entry in the **storageclass.yaml** file.

10.5.3. Available dynamic provisioning plug-ins

OpenShift Data Foundation provides the following provisioner plug-ins, which have generic implementations for dynamic provisioning that use the cluster's configured provider's API to create new storage resources:

Storage type	Provisioner plug-in name	Notes
OpenStack Cinder	**kubernetes.io/cinder**	

Storage type	Provisioner plug-in name	Notes
AWS Elastic Block Store (EBS)	**kubernetes.io/aws-ebs**	For dynamic provisioning when using multiple clusters in different zones, tag each node with **Key=kubernetes.io/cluster/<cluster_name>,Value=<cluster_id>** where **<cluster_name>** and **<cluster_id>** are unique per cluster.
AWS Elastic File System (EFS)		Dynamic provisioning is accomplished through the EFS provisioner pod and not through a provisioner plug-in.
Azure Disk	**kubernetes.io/azure-disk**	
Azure File	**kubernetes.io/azure-file**	The **persistent-volume-binder** ServiceAccount requires permissions to create and get Secrets to store the Azure storage account and keys.
GCE Persistent Disk (gcePD)	**kubernetes.io/gce-pd**	In multi-zone configurations, it is advisable to run one OpenShift Data Foundation cluster per GCE project to avoid PVs from being created in zones where no node in the current cluster exists.
VMware vSphere	**kubernetes.io/vsphere-volume**	
Red Hat Virtualization	**csi.ovirt.org**	



IMPORTANT

Any chosen provisioner plug-in also requires configuration for the relevant cloud, host, or third-party provider as per the relevant documentation.

CHAPTER 11. RECLAIMING SPACE ON TARGET VOLUMES

The deleted files or chunks of zero data sometimes take up storage space on the Ceph cluster resulting in inaccurate reporting of the available storage space. The reclaim space operation removes such discrepancies by executing the following operations on the target volume:

- **fstrim** - This operation is used on volumes that are in **Filesystem** mode and only if the volume is mounted to a pod at the time of execution of reclaim space operation.
- **rbd sparsify** - This operation is used when the volume is not attached to any pods and reclaims the space occupied by chunks of 4M-sized zeroed data.



NOTE

- Only the Ceph RBD volumes support the reclaim space operation.
- The reclaim space operation involves a performance penalty when it is being executed.

You can use one of the following methods to reclaim the space:

- Enabling reclaim space operation using annotating PersistentVolumeClaims (recommended)
- Enabling reclaim space operation using ReclaimSpaceJob
- Enabling reclaim space operation using ReclaimSpaceCronJob

In addition to reclaiming space on volumes, you can also clean up orphaned CephFS snapshots and stale subvolumes to optimize storage utilization.

Additional resources

- fstrim man page
- Enabling reclaim space operation using annotating PersistentVolumeClaims
- Enabling reclaim space operation using ReclaimSpaceJob
- Enabling reclaim space operation using ReclaimSpaceCronJob
- Finding and cleaning stale subvolumes
- Cleaning orphaned CephFS snapshots

11.1. ENABLING RECLAIM SPACE OPERATION BY ANNOTATING PERSISTENTVOLUMECLAIMS

Use this procedure to automatically invoke the reclaim space operation to annotate persistent volume claim (PVC) based on a given schedule.



NOTE

- The schedule value is in the same format as the Kubernetes CronJobs which sets the **and/or** interval of the recurring operation request.
- Recommended schedule interval is **@weekly**. If the schedule interval value is empty or in an invalid format, then the default schedule value is set to **@weekly**. Do not schedule multiple **ReclaimSpace** operations **@weekly** or at the same time.
- Minimum supported interval between each scheduled operation is at least 24 hours. For example, **@daily** (At 00:00 every day) or **0 3 * * *** (At 3:00 every day).
- Schedule the **ReclaimSpace** operation during off-peak, maintenance window, or the interval when the workload **input/output** is expected to be low.
- **ReclaimSpaceCronJob** is recreated when the **schedule** is modified. It is automatically deleted when the annotation is removed.

Procedure

1. Get the PVC details.

```
$ oc get pvc data-pvc
```

```
NAME      STATUS   VOLUME                                     CAPACITY   ACCESS MODES
STORAGECLASS                  AGE
data-pvc  Bound    pvc-f37b8582-4b04-4676-88dd-e1b95c6abf74   1Gi        RWO            ocs-
storagecluster-ceph-rbd       20h
```

2. Add annotation **reclaimspace.csiaddons.openshift.io/schedule=@monthly** to the PVC to create **reclaimspacecronjob**.

```
$ oc annotate pvc data-pvc "reclaimspace.csiaddons.openshift.io/schedule=@monthly"
```

```
persistentvolumeclaim/data-pvc annotated
```

3. Verify that **reclaimspacecronjob** is created in the format, **"<pvc-name>-xxxxxxx"**.

```
$ oc get reclaimspacecronjobs.csiaddons.openshift.io
```

```
NAME                 SCHEDULE    SUSPEND   ACTIVE   LASTSCHEDULE   AGE
data-pvc-1642663516  @monthly                       3s
```

4. Modify the schedule to run this job automatically.

```
$ oc annotate pvc data-pvc "reclaimspace.csiaddons.openshift.io/schedule=@weekly" --
overwrite=true
```

```
persistentvolumeclaim/data-pvc annotated
```

5. Verify that the schedule for **reclaimspacecronjob** has been modified.

```
$ oc get reclaimspacecronjobs.csiaddons.openshift.io
```

```
NAME                SCHEDULE    SUSPEND   ACTIVE   LASTSCHEDULE   AGE
data-pvc-1642664617   @weekly                                3s
```

11.2. DISABLING RECLAIM SPACE FOR A SPECIFIC PERSISTENTVOLUMECLAIM

To disable reclaim space for a specific PersistentVolumeClaim (PVC), modify the associated **ReclaimSpaceCronJob** custom resource (CR).

Procedure

1. Identify the **ReclaimSpaceCronJob CR** for the PVC you want to disable reclaim space on:

```
$ oc get reclaimspacecronjobs -o jsonpath='{range .items[?
(@.spec.jobTemplate.spec.target.persistentVolumeClaim=="<PVC_NAME>")]}
{.metadata.name}{"\n"}{end}'
```

   Replace **"<PVC_NAME>"** with the name of the PVC.

2. Apply the following to the **ReclaimSpaceCronJob** CR from step 1 to disable the reclaim space:

   a. Update the **csiaddons.openshift.io/state** annotation from **"managed"** to **"unmanaged"**

```
$ oc annotate reclaimspacecronjobs <RECLAIMSPACECRONJOB_NAME>
"csiaddons.openshift.io/state=unmanaged" --overwrite=true
```

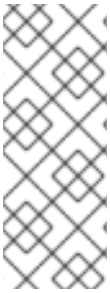
      Replace **<RECLAIMSPACECRONJOB_NAME>** with the **ReclaimSpaceCronJob** CR.

   b. Add **suspend: true** under the **spec** field:

```
$ oc patch reclaimspacecronjobs <RECLAIMSPACECRONJOB_NAME> -p '{"spec":
{"suspend": true}}' --type=merge
```

11.3. ENABLING RECLAIM SPACE OPERATION USING RECLAIMSPACEJOB

ReclaimSpaceJob is a namespaced custom resource (CR) designed to invoke reclaim space operation on the target volume. This is a one time method that immediately starts the reclaim space operation. You have to repeat the creation of **ReclaimSpaceJob** CR to repeat the reclaim space operation when required.



NOTE

- Recommended interval between the reclaim space operations is **weekly**.
- Ensure that the minimum interval between each operation is at least **24 hours**.
- Schedule the reclaim space operation during off-peak, maintenance window, or when the workload input/output is expected to be low.

Procedure

1. Create and apply the following custom resource for reclaim space operation:

```
apiVersion: csiaddons.openshift.io/v1alpha1
kind: ReclaimSpaceJob
metadata:
  name: sample-1
spec:
  target:
    persistentVolumeClaim: pvc-1
  timeout: 360
```

where,

target

Indicates the volume target on which the operation is performed.

persistentVolumeClaim

Name of the **PersistentVolumeClaim**.

backOfflimit

Specifies the maximum number of retries before marking the reclaim space operation as **failed**. The default value is **6**. The allowed maximum and minimum values are **60** and **0** respectively.

retryDeadlineSeconds

Specifies the maximum duration, in seconds, during which a **ReclaimSpaceJob** can retry after its start time. The value must be a positive integer. The default is **600** seconds, and the maximum allowed value is **1800** seconds.

timeout

Specifies the timeout in seconds for the **grpc** request sent to the CSI driver. If the timeout value is not specified, it defaults to the value of global reclaimspace timeout. Minimum allowed value for timeout is 60.

2. Delete the custom resource after completion of the operation.

11.4. ENABLING RECLAIM SPACE OPERATION USING RECLAIMSPACECRONJOB

ReclaimSpaceCronJob invokes the reclaim space operation based on the given schedule such as daily, weekly, and so on. You have to create **ReclaimSpaceCronJob** only once for a persistent volume claim. The **CSI-addons** controller creates a **ReclaimSpaceJob** at the requested time and interval with the schedule attribute.



NOTE

- Recommended schedule interval is **@weekly**.
- Minimum interval between each scheduled operation should be at least 24 hours. For example, **@daily** (At 00:00 every day) or “0 3 * * *” (At 3:00 every day).
- Schedule the ReclaimSpace operation during off-peak, maintenance window, or the interval when workload input/output is expected to be low.
- When any schedule for ReclaimSpace is used, including **@daily**, **@weekly**, **@monthly**, and cron expressions such as **0 * * * ***, multiple CronJobs are created. To prevent all Jobs from running simultaneously and overloading high-scale systems, executions are staggered within a 2-hour window from the scheduled start time (e.g., **@weekly** would be 00:00, 00:05, 00:10, and so on).
  - If the configured schedule interval is less than 2 hours, the stagger window is reduced to match the interval. For example, a job running every 30 minutes will be staggered within a 00:00–00:30 window rather than 00:00–02:00.

Procedure

1. Create and apply the following custom resource for reclaim space operation

```
apiVersion: csiaddons.openshift.io/v1alpha1
kind: ReclaimSpaceCronJob
metadata:
  name: reclaimspacecronjob-sample
spec:
  jobTemplate:
    spec:
      backOffLimit: 6
      retryDeadlineSeconds: 600
      target:
        persistentVolumeClaim: data-pvc
      timeout: 360
  schedule: '@weekly'
  concurrencyPolicy: Forbid
```

where,

failedJobsHistoryLimit

Specifies the number of failed **ReclaimSpaceJobs** that are kept for troubleshooting.

jobTemplate

Specifies the **ReclaimSpaceJob.spec** structure that describes the details of the requested **ReclaimSpaceJob** operation.

backOffLimit

Specifies the number of retry attempts for a failed **ReclaimSpaceJob** before it is marked as failed. The default value is **6**. The allowed maximum and minimum values are **60** and **0** respectively.

retryDeadlineSeconds

Specifies the maximum duration, in seconds, during which a **ReclaimSpaceJob** can retry after its start time. The value must be a positive integer. The default is **600** seconds, and the maximum allowed value is **1800** seconds.

successfulJobsHistoryLimit

Specifies the number of successful **ReclaimSpaceJob** operations.

schedule

Specifies the and/or interval of the recurring operation request and it is in the same format as the Kubernetes CronJobs.

concurrencyPolicy

Describes the changes when a new **ReclaimSpaceJob** is scheduled by the **ReclaimSpaceCronJob**, while a previous **ReclaimSpaceJob** is still running. The default **Forbid** prevents starting a new job whereas **Replace** can be used to delete the running job potentially in a failure state and create a new one.

2. Delete the **ReclaimSpaceCronJob** custom resource when execution of reclaim space operation is no longer needed or when the target PVC is deleted.

11.5. CUSTOMISING TIMEOUTS REQUIRED FOR RECLAIM SPACE OPERATION

Depending on the RBD volume size and its data pattern, Reclaim Space Operation might fail with the **context deadline exceeded** error. You can avoid this by increasing the timeout value.

The following example shows the failed status by inspecting **-o yaml** of the corresponding **ReclaimSpaceJob**:

Example

```
Status:
  Completion Time:  2023-03-08T18:56:18Z
  Conditions:
    Last Transition Time:  2023-03-08T18:56:18Z
    Message:               Failed to make controller request: context deadline exceeded
    Observed Generation:   1
    Reason:                failed
    Status:                True
    Type:                  Failed
  Message:                 Maximum retry limit reached
  Result:                  Failed
  Retries:                 6
  Start Time:              2023-03-08T18:33:55Z
```

You can also set custom timeouts at global level by creating the following **configmap**:

Example

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: csi-addons-config
  namespace: openshift-storage
data:
```

```
  "reclaim-space-timeout": "6m"
```

Restart the **csi-addons** operator pod.

```
oc delete po -n openshift-storage -l "app.kubernetes.io/name=csi-addons"
```

All Reclaim Space Operations started after the above **configmap** creation use the customized timeout.

11.6. ENFORCING PRECEDENCE FOR RECLAIM SPACE OPERATIONS

In reclaim space, precedence refers to the order in which the system checks for scheduled annotations. In OpenShift Data Foundation, the default precedence is set to storage class (recommended). This means the system reads annotations only from the storage class.

However, if you want the system to check the persistent volume claim (PVC) first and then fall back to the storage class, you can configure this behavior by setting the **schedule-precedence** to PVC in the CSI-addons ConfigMap.

You can define the **ConfigMap** for csi-addons as shown below:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: csi-addons-config
  namespace: openshift-storage
data:
  "schedule-precedence": "pvc"
```

Restart the **csi-addons** operator pod for the changes to take effect:

```
oc delete po -n openshift-storage -l "app.kubernetes.io/name=csi-addons"
```

CHAPTER 12. FINDING AND CLEANING STALE SUBVOLUMES

In certain scenarios, subvolumes may become stale and lack an associated Kubernetes (k8s) reference. These orphaned subvolumes serve no functional purpose and can be safely removed to optimize storage utilization. You can identify and delete stale subvolumes using the ODF CLI tool. This process ensures that unused resources are cleaned up, maintaining a healthy and efficient storage environment.

Prerequisites

- Download the OpenShift Data Foundation command line interface (ODF CLI) tool. With the ODF CLI tool, you can effectively manage and troubleshoot your Data Foundation environment from a terminal. You can find a compatible version and download the CLI tool from the customer portal.

Procedure

1. Find the stale subvolumes by using the **--stale** flag with the **subvolumes** command:

```
$ odf subvolume ls --stale
```

Example output:

```
# Filesystem  Subvolume Subvolumegroup State
# ocs-storagecluster-cephfilesystem csi-vol-427774b4-340b-11ed-8d66-0242ac110004 csi
stale
# ocs-storagecluster-cephfilesystem csi-vol-427774b4-340b-11ed-8d66-0242ac110005 csi
stale
```

2. Delete the stale subvolumes:

```
$ odf subvolume delete <filesystem>  <subvolumes> <subvolumegroup>
```

Replace **<subvolumes>** with a comma separated list of subvolumes from the output of the first command. The subvolumes must be of the same filesystem and subvolumegroup.

Replace **<filesystem>** and **<subvolumegroup>** with the filesystem and subvolumegroup from the output of the first command.

For example:

```
$ odf subvolume delete ocs-storagecluster-cephfilesystem csi-vol-5d836837-7d6d-404c-
b3e3-027927510701  csi
```

12.1. CLEANING ORPHANED CEPHFS SNAPSHOTS

Orphaned CephFS snapshots can accumulate over time when volume snapshots are deleted but their underlying CephFS snapshots remain. These orphaned snapshots consume storage space and can impact cluster performance. You can identify and clean up orphaned CephFS snapshots using the ODF CLI tool.

Prerequisites

- Download the OpenShift Data Foundation command line interface (ODF CLI) tool. With the

ODF CLI tool, you can effectively manage and troubleshoot your Data Foundation environment from a terminal. You can find a compatible version and download the CLI tool from the customer portal.

- Access to the OpenShift cluster with appropriate permissions.

Procedure

1. List all CephFS snapshots to identify orphaned snapshots:

```
$ odf cephfs-snap ls
```

Example output:

```
# Filesystem                    Subvolume                       Snapshot
State
# ocs-storagecluster-cephfilesystem    csi-vol-427774b4-340b-11ed-8d66-0242ac110004 csi-snap-123e4567-e89b-12d3-a456-426614174000 orphaned
# ocs-storagecluster-cephfilesystem    csi-vol-427774b4-340b-11ed-8d66-0242ac110005 csi-snap-223e4567-e89b-12d3-a456-426614174001 bound
```

2. List only orphaned CephFS snapshots using the **--orphaned** flag:

```
$ odf cephfs-snap ls --orphaned
```

This command filters the output to show only snapshots that are orphaned and no longer have a corresponding Kubernetes VolumeSnapshot object.

3. Delete orphaned CephFS snapshots:

```
$ odf cephfs-snap delete <subvolume> <snapshot>
```

For example:

```
$ odf cephfs-snap delete ocs-storagecluster-cephfilesystem csi-vol-427774b4-340b-11ed-8d66-0242ac110004 csi-snap-123e4567-e89b-12d3-a456-426614174000
```

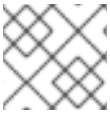
CHAPTER 13. VOLUME SNAPSHOTS

A volume snapshot is the state of the storage volume in a cluster at a particular point in time. These snapshots help to use storage more efficiently by not having to make a full copy each time and can be used as building blocks for developing an application.

Volume snapshot class allows an administrator to specify different attributes belonging to a volume snapshot object. The OpenShift Data Foundation operator installs default volume snapshot classes depending on the platform in use. The operator owns and controls these default volume snapshot classes and they cannot be deleted or modified.

You can create many snapshots of the same persistent volume claim (PVC) but cannot schedule periodic creation of snapshots.

- For CephFS, you can create up to 100 snapshots per PVC.
- For RADOS Block Device (RBD), you can create up to 512 snapshots per PVC.



NOTE

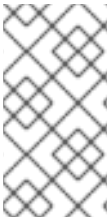
Persistent Volume encryption now supports volume snapshots.

13.1. CREATING VOLUME SNAPSHOTS

You can create a volume snapshot either from the Persistent Volume Claim (PVC) page or the Volume Snapshots page.

Prerequisites

- For a consistent snapshot, the PVC should be in **Bound** state and not be in use. Ensure to stop all IO before taking the snapshot.



NOTE

OpenShift Data Foundation only provides crash consistency for a volume snapshot of a PVC if a pod is using it. For application consistency, be sure to first tear down a running pod to ensure consistent snapshots or use any quiesce mechanism provided by the application to ensure it.

Procedure

1. From the Persistent Volume Claims page:
   a. Click **Storage → Persistent Volume Claims** from the OpenShift Web Console.
2. To create a volume snapshot, do one of the following:
   - Beside the desired PVC, click Action menu **(⋮) → Create Snapshot**.
   - Click on the PVC for which you want to create the snapshot and click **Actions → Create Snapshot**.
3. Enter a **Name** for the volume snapshot.
4. Choose the **Snapshot Class** from the drop-down list.

5. Click **Create**. You will be redirected to the Details page of the volume snapshot that is created.

   From the Volume Snapshots page

6. Click **Storage → Volume Snapshots** from the OpenShift Web Console.
7. In the **Volume Snapshots** page, click **Create Volume Snapshot**
8. Choose the required **Project** from the drop-down list.
9. Choose the **Persistent Volume Claim** from the drop-down list.
10. Enter a **Name** for the snapshot.
11. Choose the **Snapshot Class** from the drop-down list.
12. Click **Create**. You will be redirected to the Details page of the volume snapshot that is created.

Verification

- Go to the **Details** page of the PVC and click the **Volume Snapshots** tab to see the list of volume snapshots. Verify that the new volume snapshot is listed.
- Click **Storage → Volume Snapshots** from the OpenShift Web Console. Verify that the new volume snapshot is listed.
- Wait for the volume snapshot to be in **Ready** state.

13.2. RESTORING VOLUME SNAPSHOTS

When you restore a volume snapshot, a new Persistent Volume Claim (PVC) gets created. The restored PVC is independent of the volume snapshot and the parent PVC.

You can restore a volume snapshot from either the Persistent Volume Claim page or the Volume Snapshots page.

Procedure

1. From the Persistent Volume Claims page:
   You can restore volume snapshot from this page only if the parent PVC is present.

   a. Click **Storage → Persistent Volume Claims** from the OpenShift Web Console.

2. Click on the PVC name with the volume snapshot to restore a volume snapshot as a new PVC.
3. In the **Volume Snapshots** tab, click the Action menu (⋮) next to the volume snapshot you want to restore.
4. Click **Restore as new PVC**.
5. Enter a name for the new PVC.
6. Select the **Storage Class** name.
7. Select the **Access Mode** of your choice.

8. Optional: For RBD, select **Volume mode**.
9. Click **Restore**. You are redirected to the new PVC details page.

From the Volume Snapshots page

10. Click **Storage → Volume Snapshots** from the OpenShift Web Console.
11. In the **Volume Snapshots** tab, click the Action menu (⋮) next to the volume snapshot you want to restore.
12. Click **Restore as new PVC**.
13. Enter a name for the new PVC.
14. Select the **Storage Class** name.
15. Select the **Access Mode** of your choice.
16. Optional: For RBD, select **Volume mode**.
17. Click **Restore**. You are redirected to the new PVC details page.

Verification

- Click **Storage → Persistent Volume Claims** from the OpenShift Web Console and confirm that the new PVC is listed in the **Persistent Volume Claims** page.
- Wait for the new PVC to reach **Bound** state.

13.3. DELETING VOLUME SNAPSHOTS

Prerequisites

- For deleting a volume snapshot, the volume snapshot class which is used in that particular volume snapshot should be present.

Procedure

1. From the Persistent Volume Claims page:
   a. Click **Storage → Persistent Volume Claims** from the OpenShift Web Console.
2. Click on the PVC name which has the volume snapshot that needs to be deleted.
3. In the **Volume Snapshots** tab, beside the desired volume snapshot, click Action menu **(⋮) → Delete Volume Snapshot**

From Volume Snapshots page

4. Click **Storage → Volume Snapshots** from the OpenShift Web Console.
5. In the **Volume Snapshots** page, beside the desired volume snapshot click Action menu **(⋮) → Delete Volume Snapshot**

Verification

- Ensure that the deleted volume snapshot is not present in the **Volume Snapshots** tab of the PVC details page.
- Click **Storage → Volume Snapshots** and ensure that the deleted volume snapshot is not listed.

CHAPTER 14. VOLUME CLONING

Volume cloning creates a duplicate of an existing persistent volume claim at a point in time, giving you an independent copy of the data without requiring a full storage backup.

A clone is a duplicate of an existing storage volume that is used as any standard volume. You create a clone of a volume to make a point in time copy of the data. A persistent volume claim (PVC) cannot be cloned with a different size. You can create up to 512 clones per PVC for both CephFS and RADOS Block Device (RBD).

14.1. CREATING A CLONE

Prerequisites

- Source PVC must be in **Bound** state and must not be in use.



NOTE

Do not create a clone of a PVC if a Pod is using it. Doing so might cause data corruption because the PVC is not quiesced (paused).

Procedure

1. Click **Storage → Persistent Volume Claims** from the OpenShift Web Console.
2. To create a clone, do one of the following:
   - Beside the desired PVC, click Action menu **(⋮) → Clone PVC**.
   - Click on the PVC that you want to clone and click **Actions → Clone PVC**.
3. Enter a **Name** for the clone.
4. Select the access mode of your choice.
5. Enter the required size of the clone.
6. Select the storage class in which you want to create the clone.
   The storage class can be any RBD storage class and it need not necessarily be the same as the parent PVC.
7. Click **Clone**. You are redirected to the new PVC details page.
8. Wait for the cloned PVC status to become **Bound**.
   The cloned PVC is now available to be consumed by the pods. This cloned PVC is independent of its dataSource PVC.

CHAPTER 15. MANAGING CONTAINER STORAGE INTERFACE (CSI) COMPONENT PLACEMENTS

Each cluster consists of a number of dedicated nodes such as **infra** and **storage** nodes. An **infra** node with a custom taint cannot use OpenShift Data Foundation Persistent Volume Claims (PVCs) on the node unless the required tolerations are added. To use such nodes, configure tolerations to bring up **csi-plugins** on the nodes. For more information, see the knowledgebase article How to add toleration for the "non-ocs" taints to the OpenShift Data Foundation pods.

CHAPTER 16. USING 2-WAY REPLICATION WITH CEPHFS

To reduce storage overhead with CephFS when data resiliency is not a primary concern, you can opt for using 2-way replication (replica-2). This reduces the amount of storage space used and decreases the level of fault tolerance.

There are two ways to use replica-2 for CephFS:

- Edit the existing default pool to replica-2 and use it with the default CephFS **storageclass**.
- Add an additional CephFS data pool with replica-2.

Additional resources

- Edit the existing default pool to replica-2 and use it with the default CephFS **storageclass**
- Add an additional CephFS data pool with replica-2

16.1. EDITING THE EXISTING DEFAULT CEPHFS DATA POOL TO REPLICA-2

Use this procedure to edit the existing default CephFS pool to replica-2 and use it with the default CephFS storageclass.

Procedure

1. Patch the storagecluster to change default CephFS data pool to replica-2.

```
$ oc patch storagecluster ocs-storagecluster -n openshift-storage --type json --patch '[{ "op": "replace", "path": "/spec/managedResources/cephFilesystems/dataPoolSpec/replicated/size", "value": 2 }]'
storagecluster.ocs.openshift.io/ocs-storagecluster patched
```

```
$ oc get cephfilesystem ocs-storagecluster-cephfilesystem -o=jsonpath='{.spec.dataPools}' | jq
[
  {
    "application": "",
    "deviceClass": "ssd",
    "erasureCoded": {
      "codingChunks": 0,
      "dataChunks": 0
    },
    "failureDomain": "zone",
    "mirroring": {},
    "quotas": {},
    "replicated": {
      "replicasPerFailureDomain": 1,
      "size": 2,
      "targetSizeRatio": 0.49
    },
    "statusCheck": {
      "mirror": {}
```

```
    }
  }
]
```

2. Check the pool details.

```
$ ceph osd pool ls | grep filesystem
ocs-storagecluster-cephfilesystem-metadata
ocs-storagecluster-cephfilesystem-data0
```

16.2. ADDING AN ADDITIONAL CEPHFS DATA POOL WITH REPLICA-2

Use this procedure to add an additional CephFS data pool with replica-2.

Prerequisites

- Ensure that you are logged into the OpenShift Container Platform web console and OpenShift Data Foundation cluster is in **Ready** state.

Procedure

1. Click **Storage → StorageClasses → Create Storage Class**
2. Select **CephFS Provisioner**.
3. Under **Storage Pool**, click **Create new storage pool**.
   a. Fill in the **Create Storage Pool** fields.
   b. Under **Data protection policy**, select **2-way Replication**.
   c. Confirm Storage Pool creation
4. In the Storage Class creation form, choose the newly created Storage Pool.
5. Confirm the Storage Class creation.

Verification

1. Click **Storage → Data Foundation**.
2. In the **Storage systems** tab, select the new storage system.
3. The **Details** tab of the storage system reflect the correct volume and device types you chose during creation

CHAPTER 17. CREATING EXPORTS USING NFS

Create NFS exports backed by OpenShift Data Foundation to share persistent storage with applications running inside or outside your OpenShift cluster.

Follow the instructions in this section to create exports and access them externally from the OpenShift cluster:

- Enabling the NFS feature
- Creating NFS exports
- Consuming NFS exports in-cluster
- Consuming NFS exports in OpenShift cluster

Additional resources

- Section 17.1, “Enabling the NFS feature”
- Section 17.2, “Creating NFS exports”
- Section 17.3, “Consuming NFS exports in-cluster”
- Section 17.4, “Consuming NFS exports in OpenShift cluster”

17.1. ENABLING THE NFS FEATURE

To use NFS feature, you need to enable it in the storage cluster using the command-line interface (CLI) after the cluster is created. You can also enable the NFS feature while creating the storage cluster using the user interface.

Prerequisites

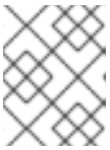
- OpenShift Data Foundation is installed and running in the **openshift-storage** namespace.
- The OpenShift Data Foundation installation includes a CephFilesystem.

Procedure

1. Run the following command to enable the NFS feature from CLI:

```
$ oc --namespace openshift-storage patch storageclusters.ocs.openshift.io ocs-storagecluster --type merge --patch '{"spec": {"nfs":{"enable": true}}}'
```

2. Specify a custom NFS Ganesha server for NFS StorageClasses:
   You can optionally set **externalEndpoint** in **StorageCluster.spec.nfs** to define the externally resolvable IP address or DNS name of the NFS Ganesha server to be used by NFS StorageClass.



NOTE

When configured, internal clients will also use this address instead of the default NFS service name.

```
oc --namespace openshift-storage patch storageclusters.ocs.openshift.io ocs-storagecluster \
  --type=merge \
  --patch '{"spec":{"nfs":{"externalEndpoint":"<VALUE>"}}}'
```

Verification

NFS installation and configuration is complete when the following conditions are met:

- The CephNFS resource named **ocs-storagecluster-cephnfs** has a status of **Ready**.
- Check if all the **csi-nfsplugin-*** pods are running:

  ```
  oc -n openshift-storage describe cephnfs ocs-storagecluster-cephnfs
  ```

  ```
  oc -n openshift-storage get pod | grep csi-nfsplugin
  ```

  Output has multiple pods. For example:

  ```
  csi-nfsplugin-47qwq                                2/2     Running  0  10s
  csi-nfsplugin-77947                                2/2     Running  0  10s
  csi-nfsplugin-ct2pm                                2/2     Running  0  10s
  csi-nfsplugin-provisioner-f85b75fbb-2rm2w          2/2     Running  0  10s
  csi-nfsplugin-provisioner-f85b75fbb-8nj5h          2/2     Running  0  10s
  ```

17.2. CREATING NFS EXPORTS

NFS exports are created by creating a Persistent Volume Claim (PVC) against the **ocs-storagecluster-ceph-nfs** StorageClass.

You can create NFS PVCs two ways:

Create NFS PVC using a yaml.

The following is an example PVC.



NOTE

volumeMode: Block will not work for NFS volumes.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: <desired_name>
spec:
 accessModes:
   - ReadWriteOnce
 resources:
   requests:
     storage: 1Gi
 storageClassName: ocs-storagecluster-ceph-nfs
```

<desired_name>

Specify a name for the PVC, for example, **my-nfs-export**.

The export is created once the PVC reaches the **Bound** state.

Create NFS PVCs from the OpenShift Container Platform web console.

Prerequisites

- Ensure that you are logged into the OpenShift Container Platform web console and the NFS feature is enabled for the storage cluster.

Procedure

1. In the OpenShift Web Console, click **Storage → Persistent Volume Claims**
2. Set the **Project** to **openshift-storage**.
3. Click **Create PersistentVolumeClaim**.
   a. Specify **Storage Class**, **ocs-storagecluster-ceph-nfs**.
   b. Specify the **PVC Name**, for example, **my-nfs-export**.
   c. Select the required **Access Mode**.
   d. Specify a **Size** as per application requirement.
   e. Select **Volume mode** as **Filesystem**.
   Note: **Block** mode is not supported for NFS PVCs
   f. Click **Create** and wait until the PVC is in **Bound** status.

17.3. CONSUMING NFS EXPORTS IN-CLUSTER

Kubernetes application pods can consume NFS exports created by mounting a previously created PVC.

You can mount the PVC one of two ways:

Using a YAML:

Below is an example pod that uses the example PVC created in Section 17.2, “Creating NFS exports”:

```
apiVersion: v1
kind: Pod
metadata:
 name: nfs-export-example
spec:
 containers:
   - name: web-server
     image: nginx
     volumeMounts:
       - name: nfs-export-pvc
         mountPath: /var/lib/www/html
 volumes:
   - name: nfs-export-pvc
```

```
  persistentVolumeClaim:
    claimName: <pvc_name>
    readOnly: false
```

<pvc_name>

Specify the PVC you have previously created, for example, **my-nfs-export**.

Using the OpenShift Container Platform web console.

Procedure

1. On the OpenShift Container Platform web console, navigate to **Workloads → Pods**.
2. Click **Create Pod** to create a new application pod.
3. Under the **metadata** section add a name. For example, **nfs-export-example**, with **namespace** as **openshift-storage**.
4. Under the **spec:** section, add **containers:** section with **image** and **volumeMounts** sections:

```
apiVersion: v1
kind: Pod
metadata:
 name: nfs-export-example
 namespace: openshift-storage
spec:
 containers:
   - name: web-server
     image: nginx
     volumeMounts:
       - name: <volume_name>
         mountPath: /var/lib/www/html
```

For example:

```
apiVersion: v1
kind: Pod
metadata:
 name: nfs-export-example
 namespace: openshift-storage
spec:
 containers:
   - name: web-server
     image: nginx
     volumeMounts:
       - name: nfs-export-pvc
         mountPath: /var/lib/www/html
```

5. Under the **spec:** section, add **volumes:** section to add the NFS PVC as a volume for the application pod:

```
volumes:
  - name: <volume_name>
    persistentVolumeClaim:
      claimName: <pvc_name>
```

–

For example:

```
volumes:
  - name: nfs-export-pvc
    persistentVolumeClaim:
      claimName: my-nfs-export
```

17.4. CONSUMING NFS EXPORTS IN OPENSHIFT CLUSTER

NFS clients running in containers or OpenShift Virtualization VMs within the same cluster as OpenShift Data Foundation can mount NFS exports created from an existing PVC.

Procedure

1. After the **nfs** flag is enabled, singe-server CephNFS is deployed by Rook. You need to fetch the value of the **ceph_nfs** field for the **nfs-ganesha** server to use in the next step:

```
$ oc get pods -n openshift-storage | grep rook-ceph-nfs
```

```
$ oc describe pod  <name of the rook-ceph-nfs pod> | grep ceph_nfs
```

For example:

```
$ oc describe pod rook-ceph-nfs-ocs-storagecluster-cephnfs-a-7bb484b4bf-bbdhs | grep
ceph_nfs
  ceph_nfs=my-nfs
```

2. Expose the NFS server outside of the OpenShift cluster by creating a Kubernetes LoadBalancer Service. The example below creates a LoadBalancer Service and references the NFS server created by OpenShift Data Foundation.

```
apiVersion: v1
kind: Service
metadata:
 name: rook-ceph-nfs-ocs-storagecluster-cephnfs-load-balancer
 namespace: openshift-storage
spec:
 ports:
   - name: nfs
     port: 2049
 type: LoadBalancer
 externalTrafficPolicy: Local
 selector:
   app: rook-ceph-nfs
   ceph_nfs: <my-nfs>
   instance: a
```

Replace **<my-nfs>** with the value you got in step 1.

3. Collect connection information. The information NFS clients need to connect to an export comes from the Persistent Volume (PV) created for the PVC, and the status of the LoadBalancer Service created in the previous step.

a. Get the share path from the PV.

i. Get the name of the PV associated with the NFS export's PVC:

```
$ oc get pvc <pvc_name> --output jsonpath='{.spec.volumeName}'
pvc-39c5c467-d9d3-4898-84f7-936ea52fd99d
```

Replace **<pvc_name>** with your own PVC name. For example:

```
oc get pvc pvc-39c5c467-d9d3-4898-84f7-936ea52fd99d --output
jsonpath='{.spec.volumeName}'
pvc-39c5c467-d9d3-4898-84f7-936ea52fd99d
```

ii. Use the PV name obtained previously to get the NFS export's share path:

```
$ oc get pv pvc-39c5c467-d9d3-4898-84f7-936ea52fd99d --output
jsonpath='{.spec.csi.volumeAttributes.share}'
/0001-0011-openshift-storage-0000000000000001-ba9426ab-d61b-11ec-9ffd-
0a580a800215
```

b. Get an ingress address for the NFS server. A service's ingress status may have multiple addresses. Choose the one desired to use for NFS clients. In the example below, there is only a single address: the host name **ingress-id.somedomain.com**.

```
$ oc -n openshift-storage get service rook-ceph-nfs-ocs-storagecluster-cephnfs-load-
balancer --output jsonpath='{.status.loadBalancer.ingress}'
[{"hostname":"ingress-id.somedomain.com"}]
```

4. Connect the external client using the share path and ingress address from the previous steps. The following example mounts the export to the client's directory path **/export/mount/path**:

```
$ mount -t nfs4 -o proto=tcp ingress-id.somedomain.com:/0001-0011-openshift-storage-
0000000000000001-ba9426ab-d61b-11ec-9ffd-0a580a800215 /export/mount/path
```

If this does not work immediately, it could be that the Kubernetes environment is still taking time to configure the network resources to allow ingress to the NFS server.

CHAPTER 18. ANNOTATING ENCRYPTED RBD STORAGE CLASSES

Encrypted RADOS block device RBD storage classes created before OpenShift Data Foundation 4.14 must be manually annotated with **cdi.kubevirt.io/clone-strategy=copy**. This enables customer data integration (CDI) to use host-assisted cloning instead of the default smart cloning.

The keys used to access an encrypted volume are tied to the namespace where the volume was created. When cloning an encrypted volume to a new namespace, such as, provisioning a new OpenShift Virtualization virtual machine, a new volume must be created and the content of the source volume must then be copied into the new volume. This behavior is triggered automatically if the storage class is properly annotated.

CHAPTER 19. ENABLING FASTER CLIENT IO OR RECOVERY IO DURING OSD BACKFILL

During a maintenance window, you may want to favor either client IO or recovery IO. Favoring recovery IO over client IO will significantly reduce OSD recovery time. The valid recovery profile options are **balanced**, **high_client_ops**, and **high_recovery_ops**. Set the recovery profile using the following procedure.

Prerequisites

- Download the OpenShift Data Foundation command line interface (CLI) tool. With the Data Foundation CLI tool, you can effectively manage and troubleshoot your Data Foundation environment from a terminal. You can find a compatible version and download the CLI tool from the customer portal.

Procedure

1. Check the current recovery profile:

```
$ odf get recovery-profile
```

2. Modify the recovery profile:

```
$ odf set recovery-profile <option>
```

   Replace **option** with either **balanced**, **high_client_ops**, or **high_recovery_ops**.

3. Verify the updated recovery profile:

```
$ odf get recovery-profile
```

CHAPTER 20. SETTING CEPH OSD FULL THRESHOLDS

Configure Ceph OSD full thresholds to control when the cluster warns, restricts backfill, or blocks writes as storage capacity is consumed, preventing data loss from a full cluster.

You can set Ceph OSD full thresholds using the ODF CLI tool or by updating the StorageCluster CR.

20.1. SETTING CEPH OSD FULL THRESHOLDS USING THE ODF CLI TOOL

You can set Ceph OSD full thresholds temporarily by using the ODF CLI tool. This is necessary in cases when the cluster gets into a full state and the thresholds need to be immediately increased.

Prerequisites

- Download the OpenShift Data Foundation command-line interface (CLI) tool. With the Data Foundation CLI tool, you can effectively manage and troubleshoot your Data Foundation environment from a terminal. You can find a compatible version and download the CLI tool from the customer portal.

Procedure

- Use the **set** command to adjust Ceph full thresholds. The **set** command supports the subcommands **full**, **backfillfull**, and **nearfull**.



NOTE

Ensure that the thresholds follow the correct order: **nearfull** < **backfillfull** < **full**. This means the ratio values for **nearfull** and **backfillfull** must always remain lower than the ratio of **full**.

The following examples demonstrate how to use each subcommand:

full: This subcommand allows updating the Ceph OSD full ratio in case Ceph prevents the IO operation on OSDs that reached the specified capacity. The default is **0.85**.



NOTE

If the value is set too close to **1.0**, the cluster becomes unrecoverable if the OSDs are full and there is nowhere to grow.

For example, set Ceph OSD full ratio to **0.9** and then add capacity:

```
$ odf set full 0.9
```

For instructions to add capacity for your specific use case, see the Scaling storage guide.

1. If OSDs continue to be in **stuck**, **pending**, or do not come up at all, perform the following steps:
2. Stop all IOs.

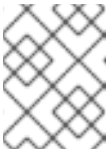
3. Increase the **full** ratio to **0.92**:

```
$ odf set full 0.92
```

4. Wait for the cluster rebalance to happen. Once cluster rebalance is complete, change the **full** ratio back to its original value of 0.85:

```
$ odf set full 0.85
```

5. **backfillfull**: This subcommand allows updating the Ceph OSDd **backfillfull** ratio in case Ceph denies backfilling to the OSD that reached the capacity specified. The default value is **0.80**.



NOTE

If the value is set too close to **1.0**, the OSDs become full and the cluster is not able to backfill.

For example, set **backfillfull** to **0.85**:

```
$ odf set backfillfull 0.85
```

6. **nearfull**: This subcommand allows updating the Ceph OSD **nearfull** ratio in case Ceph returns the **nearfull** OSDs message when the cluster reaches the capacity specified. The default value is **0.75**.
For example, set **nearfull** to **0.8**:

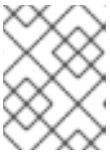
```
$ odf set nearfull 0.8
```

20.2. SETTING CEPH OSD FULL THRESHOLDS BY UPDATING THE STORAGECLUSTER CR

You can set Ceph OSD full thresholds by updating the StorageCluster CR. Use this procedure if you want to override the default settings.

Procedure

1. Update the StorageCluster CR to change the settings for **full**, **backfillfull**, and **nearfull**.
full: Update the Ceph OSD full ratio to prevent IO operations when OSDs reach the specified capacity. The default is **0.85**.



NOTE

If the value is set too close to **1.0**, the cluster becomes unrecoverable if the OSDs are full and there is nowhere to grow.

For example, to set Ceph OSD full ratio to **0.9**:

```
$ oc patch storagecluster ocs-storagecluster -n openshift-storage --type json --patch '[{ "op": "replace", "path": "/spec/managedResources/cephCluster/fullRatio", "value": 0.90 }]'
```

2. **backfillfull**: Set the Ceph OSD backfillfull ratio to deny backfilling when an OSD reaches the specified capacity. The default value is **0.80**.



NOTE

If the value is set too close to **1.0**, the OSDs become full and the cluster is not able to backfill.

For example, set backfill full to **0.85**:

```
$ oc patch storagecluster ocs-storagecluster -n openshift-storage --type json --patch '[{ "op": "replace", "path": "/spec/managedResources/cephCluster/backfillFullRatio", "value": 0.85 }]'
```

3. **nearfull**: Set the Ceph OSD nearfull ratio to return a nearfull warning when the cluster reaches the specified capacity. The default value is **0.75**.
For example, set nearfull to **0.8**:

```
$ oc patch storagecluster ocs-storagecluster -n openshift-storage --type json --patch '[{ "op": "replace", "path": "/spec/managedResources/cephCluster/nearFullRatio", "value": 0.8 }]'
```

CHAPTER 21. CONFIGURING CEPH TARGET SIZE RATIOS

Depending on the cluster usage and how you expect to fill the cluster among the three types of storage: block, shared filesystem, and object storage, you can set a value to the **targetSizeRatio** parameter. The target size ratio is a relative value used for balancing the data across pools. The sum of all pool target size ratios need not add up to 1 but it can be greater or lesser than 1 and Ceph adjusts the values accordingly. However, each value should be a non-zero value even though it is very small.



NOTE

Target size ratio does not mean there is an immediate change in the placement group (PG) number of pools.

Procedure

- To optimize for block storage, run the following commands and set the target ratios. In this example, block storage is optimized for 80%.

```
$ oc patch storagecluster ocs-storagecluster -n openshift-storage --type merge -p '
{
  "spec": {
    "managedResources": {
      "cephBlockPools": {
        "poolSpec": {
          "replicated": {
            "size": 3,
            "targetSizeRatio": 0.8
          }
        }
      }
    }
  }
}'
```

```
$ oc patch storagecluster ocs-storagecluster -n openshift-storage --type merge -p '
{
  "spec": {
    "managedResources": {
      "cephFilesystems": {
        "dataPoolSpec": {
          "replicated": {
            "size": 3,
            "targetSizeRatio": 0.2
          }
        }
      }
    }
  }
}'
```

```
$ ceph osd pool autoscale-status
POOL                                SIZE  TARGET SIZE  RATE  RAW CAPACITY   RATIO
TARGET RATIO  EFFECTIVE RATIO  BIAS  PG_NUM  NEW PG_NUM  AUTOSCALE
BULK
```

```
.mgr                                    580.0k              3.0        6144G  0.0000
1.0      32              on         False
ocs-storagecluster-cephblockpool          216.7M              3.0        6144G  0.8000
0.8000           0.8000   1.0      256              on         False
ocs-storagecluster-cephfilesystem-metadata  55030               3.0        6144G  0.0000
4.0      32              on         False
ocs-storagecluster-cephfilesystem-data0         0               3.0        6144G  0.2000
0.2000           0.2000   1.0      128          32  on         False
```

- To optimize storage in the filesystem, run the following commands:
  In this example, the filesystem is optimized to 98%. Block pool **targetSizeRatio**: 0.01/0.02
  Filesystem **targetSizeRatio**: 0.98 Object store **targetSizeRatio**: 0.01 (if present)

```
$ oc patch storagecluster ocs-storagecluster -n openshift-storage --type merge -p '
{
  "spec": {
    "managedResources": {
      "cephBlockPools": {
        "poolSpec": {
          "replicated": {
            "size": 3,
            "targetSizeRatio": 0.02
          }
        }
      }
    }
  }
}'
```

```
$ oc patch storagecluster ocs-storagecluster -n openshift-storage --type merge -p '
{
  "spec": {
    "managedResources": {
      "cephFilesystems": {
        "dataPoolSpec": {
          "replicated": {
            "size": 3,
            "targetSizeRatio": 0.98
          }
        }
      }
    }
  }
}'
```

```
$ ceph osd pool autoscale-status
POOL                                    SIZE  TARGET SIZE  RATE  RAW CAPACITY   RATIO
TARGET RATIO  EFFECTIVE RATIO  BIAS  PG_NUM  NEW PG_NUM  AUTOSCALE
BULK
.mgr                                    580.0k              3.0        6144G  0.0000
1.0      32              on         False
ocs-storagecluster-cephblockpool          216.9M              3.0        6144G  0.0909
0.0200           0.0909   1.0      256          32  on         False
ocs-storagecluster-cephfilesystem-metadata  59126               3.0        6144G  0.0000
4.0      32              on         False
```

```
ocs-storagecluster-cephfilesystem-data0        0                3.0        6144G  0.9091
0.2000           0.9091   1.0      32        256  on         False
```

- To optimize storage in **cephobjectstore**, run the following command:
  In this example, the **cephobjectstore** is optimized to 80%.

```
$ oc patch storagecluster ocs-storagecluster -n openshift-storage --type merge -p '
{
  "spec": {
    "managedResources": {
      "cephObjectStores": {
        "dataPoolSpec": {
          "replicated": {
            "size": 3,
            "targetSizeRatio": 0.80
          }
        }
      }
    }
  }
}'
```

- To edit target size ratios of custom **cephblockpool** data pool, run the following command:

```
$ oc patch cephblockpool ceph1-pool-block --type=json -p '[{"op": "add", "path":
"/spec/replicated/targetSizeRatio", "value":0.99}]
```

- To edit target size ratios of custom filesystem data pools, edit the **ocs-storagecluster** YAML under **managedResources** > **cephFilesystems** > **additionalDataPools** as follows:

```
    additionalDataPools:
    - compressionMode: aggressive
      name: pool-1
      replicated:
        size: 3
        targetSizeRatio: 0.80:
```

CHAPTER 22. CONFIGURING CEPH MONITOR FAILOVER TIMEOUT

The monitor failover timeout setting gives cluster administrators precise control over Ceph monitor (mon) failover behavior.

By default, the **rook-ceph** operator replaces a down monitor after a fixed 10 minute interval to maintain quorum. While this supports availability, it can sometimes be too aggressive triggering failovers during short, expected outages, or too conservative, increasing the risk of quorum loss if another monitor fails during that window.

The configurable timeout parameters allow administrators to adjust this behavior or disable automatic monitor failover entirely. This flexibility is particularly useful during maintenance operations, such as node drains or reboots, when temporarily preventing monitor failover helps avoid unnecessary disruption. This provides more flexibility and control over monitor recovery behavior, simplifying maintenance, and improving stability during planned operations.

Parameters for monitor health check

Parameter	Description	Default	Comments
disabled	Boolean flag to enable or disable the monitor health check/failover mechanism.	false (enabled) ,dlmslmvdmv	By default, automatic monitor failover is enabled. **Set this parameter to true only if you want to disable the monitor failover.**
interval	Specifies how long to wait between iterations	45s	
timeout	Specifies how long the operator should wait for an unhealthy Ceph monitor to recover before triggering a failover.	10m	Default is 15m for IBM Cloud

These parameters are exposed in the **StorageCluster** CR, eliminating the need to configure them in the **CephCluster** CR manually. This simplifies management and keeps configurations consistent across cluster updates.

Example configuration

The following is an example of configuring the monitor failover timeout and disabling it directly in the **StorageCluster** CR:

```
apiVersion: ocs.openshift.io/v1
kind: StorageCluster
metadata:
  name: ocs-storagecluster
  namespace: openshift-storage
```

```
spec:
  managedResources:
    cephCluster:
      healthCheck:
        daemonHealth:
          mon:
            disabled: false # Ensure automatic monitor failover is enabled (default)
            interval: 30s   # Wait 30 seconds between iterations
            timeout: 15m    # Wait 15 minutes before failing over a down monitor
```

In this example, the failover timeout is increased to 15 minutes (from the default 10), and failover remains enabled.

To disable failover entirely, set **disabled: true** in the **StorageCluster** CR.

CHAPTER 23. BACKUP AND RECOVERY OF MULTICLOUD OBJECT GATEWAY METADATA DATABASE (NOOBAA DB)

The Multicloud Object Gateway supports automated and on-demand database backups to help maintain data resilience and improve recovery options. You can configure backup frequency during deployment by selecting the frequency, such as **none**, **daily**, **weekly**, or **monthly**. Also, you can define the number of retained backups (1–12). The Object dashboard displays the time of the most recent backup. In addition to scheduled backups, you can initiate on-demand backups through the MCG CLI or a dedicated custom resource. Backup schedules and retention settings can also be modified post-deployment through YAML-based configuration updates.

23.1. ENABLING BACKUP OF MULTICLOUD OBJECT GATEWAY METADATA DATABASE USING STORAGECLUSTER CR

Enable automatic backup of the Multicloud Object Gateway metadata database by configuring the **dbBackup** field in the StorageCluster custom resource.

Procedure

- To enable DB backup using the storagecluster CR, configure **dbBackup** in the CR under **multiCloudGateway**:

```
multiCloudGateway:
  dbBackup:
    schedule: [cron format]
    volumeSnapshot:
      maxSnapshots: [integer]# number of snapshots to retain
      volumeSnapshotClass: [the vol snapshot class] # for ceph rbd ocs-storagecluster-
rbdplugin-snapclass
```

  - The operator creates a **scheduledbackups.postgresql.cnpg.noobaa.io** object.
  - According to the schedule, the CNPG operator creates a **backups.postgresql.cnpg.noobaa.io** CR that triggers a volume snapshot of the secondary instance.

23.2. ENABLING BACKUP OF MULTICLOUD OBJECT GATEWAY METADATA DATABASE USING MCG CLI

Back up the Multicloud Object Gateway metadata database using the MCG CLI to protect NooBaa configuration and bucket data from loss.

Procedure

- To manually backup using MCG CLI, use the command:

```
noobaa system db-backup [--name custom-name]
```

  MCG CLI creates a **backups.postgresql.cnpg.noobaa.io** with the given name, or by default **noobaa-db-pg-cluster-backup-[timestamp]**. The CLI command waits for the volume snapshot to be ready before exiting. Timeout is 5 minutes.

23.3. RECOVERING MULTICLOUD OBJECT GATEWAY METADATA DATABASE FROM BACKUP

Restore the Multicloud Object Gateway metadata database from a backup by setting the **dbRecovery** property in the StorageCluster custom resource.

Procedure

- To recover the database from backup, set the **dbRecovery** property in the storagecluster CR:

```
multiCloudGateway:
 dbRecovery:
   volumeSnapshotName: [volume snapshot to recover]
```

  The configured volume snapshot is excluded from the retention policy and will not be deleted as long as it is set in the CR.

- To trigger the recovery, manually delete the cluster CR:

```
oc delete clusters.postgresql.cnpg.noobaa.io noobaa-db-pg-cluster
```

  After the deletion:

  1. NooBaa operator initiates the creation of a new cluster, recovering from the volume snapshot.
  2. **noobaa-core** and **noobaa-operator** pods are stopped.
  3. DB cluster is created.
  4. Core and endpoints are restarted and the system becomes ready.